

Лекция 2. Параметрический полиморфизм

ФП 2.0 · осень 2026

Андрей Стоян

осень 2026

План на сегодня

- **Полиморфизм в языке:** System F, кайнды, полиморфизм первого класса, higher-kinded, GADT, структуры на уровне типов
- **Реализация:** сколько полиморфизм стоит во время исполнения
- **Полиморфизм по конвенции вызова:** boxed и unboxed, **TYPE**, representation polymorphism

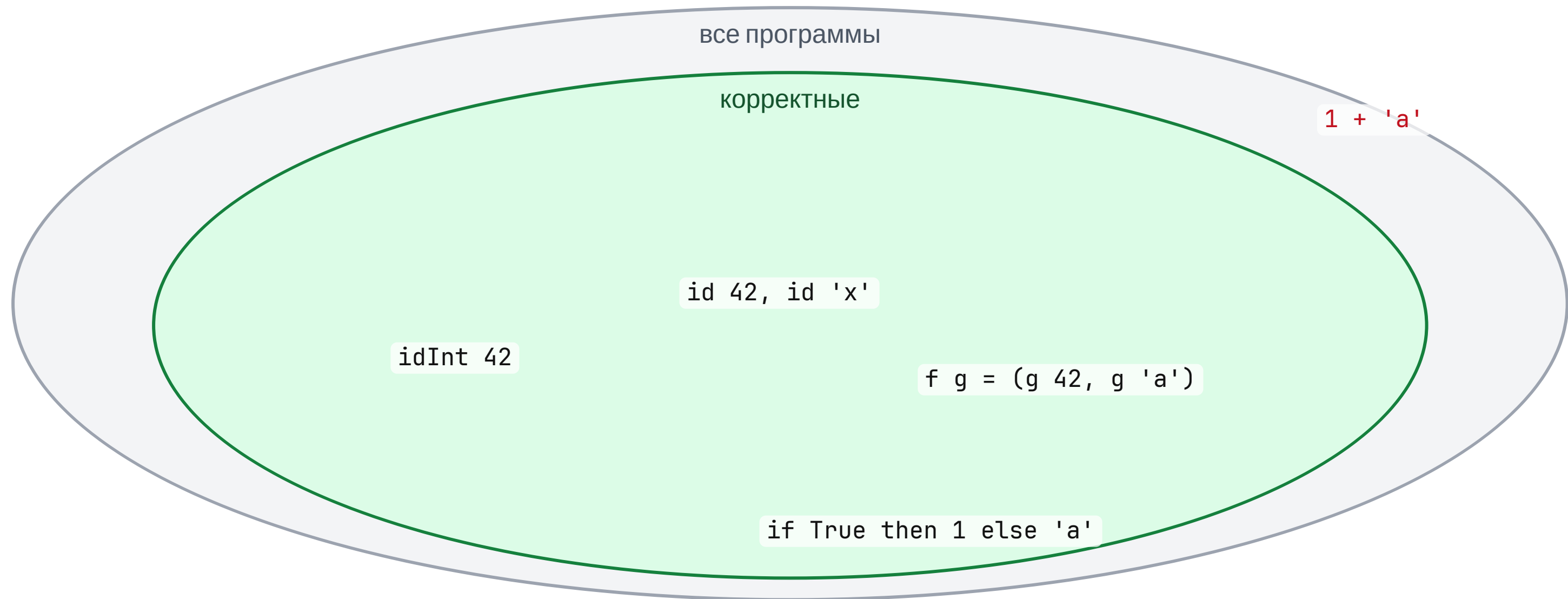
Где мы на карте

Что сегодня	Где выстрелит
универсальная аппликация, кайнды	словари классов типов и семейства типов — глава 3
higher-kinded полиморфизм	Fix и свёртки — глава 4; Free — глава 7
типы высшего ранга	tagless final — глава 5; ST , хендлеры — главы 6–7
GADT и структуры на уровне типов	типизированные интерпретаторы — глава 5; системы эффектов — глава 8
стирание типов и словари	реализация классов типов — глава 3

Параметрический полиморфизм в языке

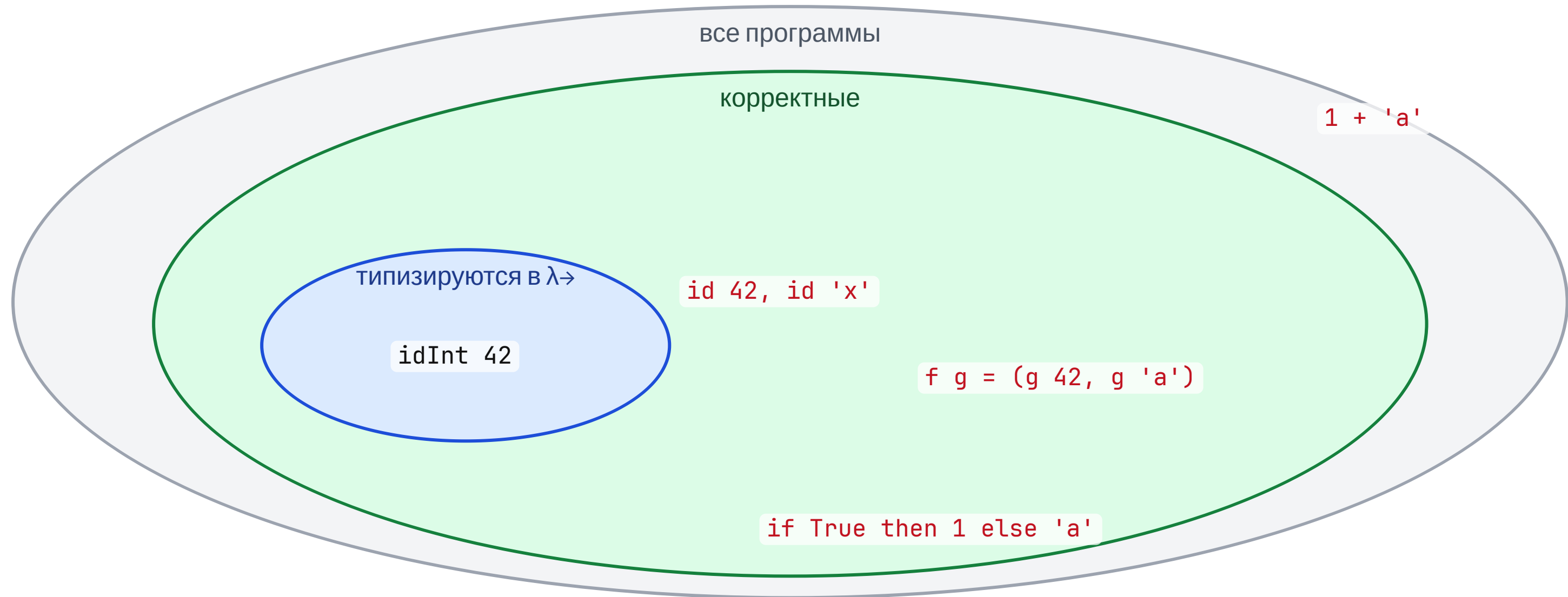
Что пропускает система типов

Корректность — семантическое свойство программы. По теореме Райса алгоритмически её границу не найти.



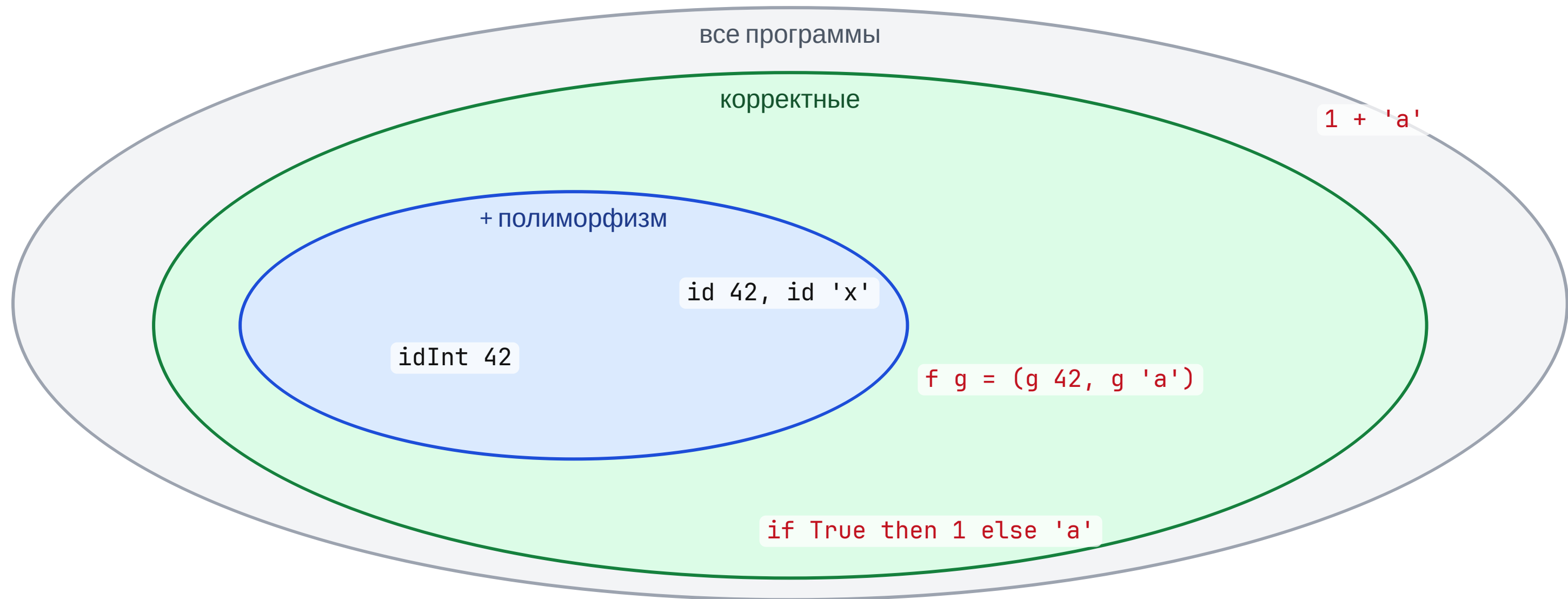
Что пропускает система типов

Разрешимая система типов очерчивает область поменьше: вместе с некорректными программами она отвергает много корректных.



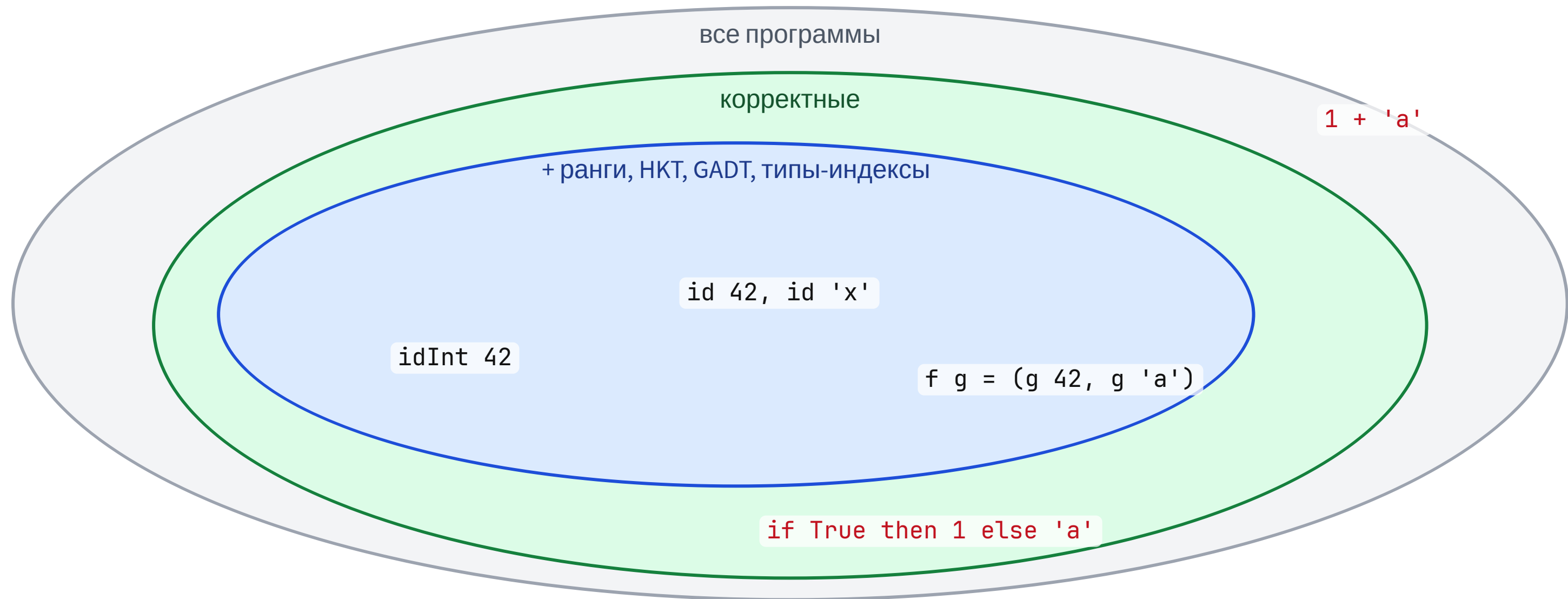
Что пропускает система типов

Параметрический полиморфизм расширяет область: один код единообразно работает с любыми типами.



Что пропускает система типов

Сегодня расширяем её дальше: ранги, higher-kinded, GADT, структуры на уровне типов. Граница всё равно останется внутри зелёной.



Скрытый аргумент

Один `id` — три места вызова. Кто и когда выбирает конкретный тип вместо `a`?

`id x = x`

`id 42`

`id 'x'`

`id True`

Скрытый аргумент

Вызывающий код, в каждом месте свой: у `id` есть ещё один, скрытый аргумент — сам тип.

```
id x = x
```

```
id @Int 42
```

```
id @Char 'x'
```

```
id @Bool True
```

Скрытый аргумент

И в определении его можно принять явно: функция двух аргументов — типа и значения.

`id @a (x :: a) = x`

`id @Int 42`

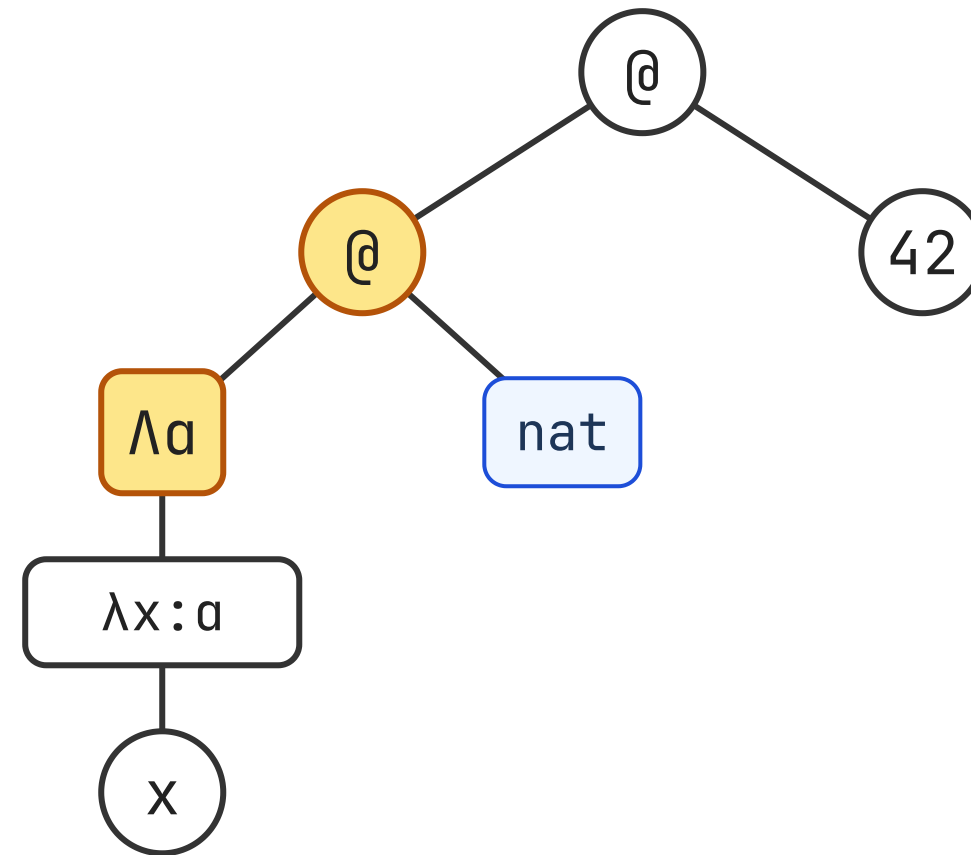
`id @Char 'x'`

`id @Bool True`

Тип — такой же аргумент

System F: $\Lambda\alpha$ обобщает выражение по типу, аппликация к типу его подставляет. Получается редекс, как у обычной функции.

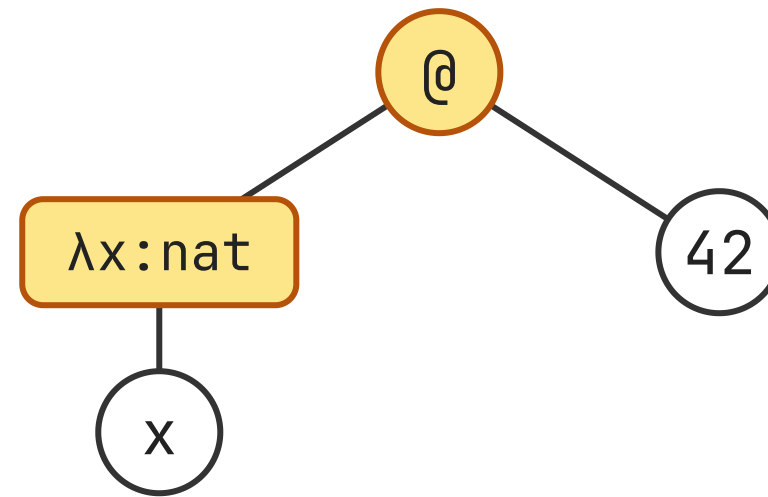
$(\Lambda\alpha. \lambda x:\alpha. x) \text{ nat } 42$



Тип — такой же аргумент

Тип подставлен в аннотацию параметра: $\alpha \mapsto \text{nat}$. Осталась обычная функция и обычный β -редекс.

$(\lambda x:\text{nat}. x) 42$



Тип — такой же аргумент

Вызов `id @Int 42` в Haskell — те же два шага: сначала выбран тип, потом передано значение.

42

42

Одна и та же пара правил — для значений и для типов

Обобщаем по значению: λ добавляет стрелку, аппликация снимает

$$\frac{x : \tau, \Gamma \vdash M : \sigma}{\Gamma \vdash \lambda x : \tau. M : \tau \rightarrow \sigma} \text{Lam}$$
$$\frac{\Gamma \vdash M : \tau \rightarrow \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash M N : \sigma} \text{App}$$

Обобщаем по типу: Λ добавляет $\forall$, аппликация к типу снимает

$$\frac{\alpha, \Gamma \vdash M : \tau}{\Gamma \vdash \Lambda \alpha. M : \forall \alpha. \tau} \text{TLam}$$
$$\frac{\Gamma \vdash M : \forall \alpha. \tau}{\Gamma \vdash M \sigma : [\alpha \mapsto \sigma] \tau} \text{TApp}$$

Дерево вывода: **id** получает тип, потом значение

id = $\Lambda\alpha. \lambda x : \alpha. x$, тип **id** : $\forall\alpha. \alpha \rightarrow \alpha$

$$\frac{\frac{\frac{}{\vdash \mathbf{id} : \forall\alpha. \alpha \rightarrow \alpha} \text{def}}{\vdash \mathbf{id} \text{ nat} : \text{nat} \rightarrow \text{nat}} \text{TApp} \quad \frac{}{\vdash 42 : \text{nat}} \text{const}}{\vdash \mathbf{id} \text{ nat} 42 : \text{nat}} \text{App}$$

ШАГИ

1. App: аппликация — две посылки
2. TApp: снимаем $\forall$, $\alpha \mapsto \text{nat}$; const: 42
3. def: тип id по определению

Когда типы приходится писать руками

```
ghci> read "42" -- какой тип результата? из термина не вывести
*** Exception: Prelude.read: no parse

ghci> read @Int "42"
42

ghci> read @Double "42"
42.0
```

✎ Какой тип у выражения `const @Int`? А у `const @Int @Char`? От чего зависит порядок, в котором передаются типы?

⚡ Решение.

```
const @Int      :: Int -> b -> Int
const @Int @Char :: Int -> Char -> Int
```

Порядок — порядок первого появления типовых переменных в сигнатуре слева направо; явный `forall b a` . его меняет.

Типы тоже вычисляются

```
type Pair a b = forall c . (a -> b -> c) -> c    -- пара по Чёрчу
```

```
pair :: a -> b -> Pair a b  
pair x y = \f -> f x y
```

```
fst :: Pair a b -> a  
fst p = p (\x _ -> x)
```

```
ghci> :kind! Pair Int Int  
= forall c. (Int -> Int -> c) -> c    -- синоним — функция на типах: вычислился
```

```
ghci> :kind! Maybe Int  
= Maybe Int    -- конструктор номинативен: вычислять нечего
```

λ-исчисление на уровне типов

Синоним с параметрами — абстракция, только на типах

$$\mathbf{Pair} = \lambda\tau\sigma. \forall\gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma$$

Применение абстракции к типу редуцируется подстановкой, как на уровне термов

$$(\lambda\alpha. \tau) \sigma \rightsquigarrow [\alpha \mapsto \sigma] \tau$$

Редукция Pair Int Int

Что произошло в GHCi? Применение синонима к аргументам — выражение на уровне типов, и его можно вычислять.

Pair Int Int

Редукция Pair Int Int

Раскрываем имя **Pair**: получаем аппликацию типовой абстракции к двум аргументам — редекс на уровне ТИПОВ.

$$(\lambda\tau\sigma. \forall\gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma) \text{ Int Int}$$

Редукция Pair Int Int

Первый β -шаг на уровне типов: $\tau \mapsto \text{Int}$; остался редекс с σ .

$$(\lambda\sigma. \forall\gamma. (\text{Int} \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma) \text{Int}$$

Редукция Pair Int Int

Второй β -шаг: $\sigma \mapsto \text{Int}$. Нормальная форма — полиморфный тип, у которого квантор остался внутри: пара — функция, ждущая продолжение.

$$\forall \gamma. (\text{Int} \rightarrow \text{Int} \rightarrow \gamma) \rightarrow \gamma$$

Раз типы применяют к типам — их можно применить неправильно

```
ghci> :kind Maybe Maybe
error: Expecting one more argument to 'Maybe'
      Expected a type, but 'Maybe' has kind '* -> *'

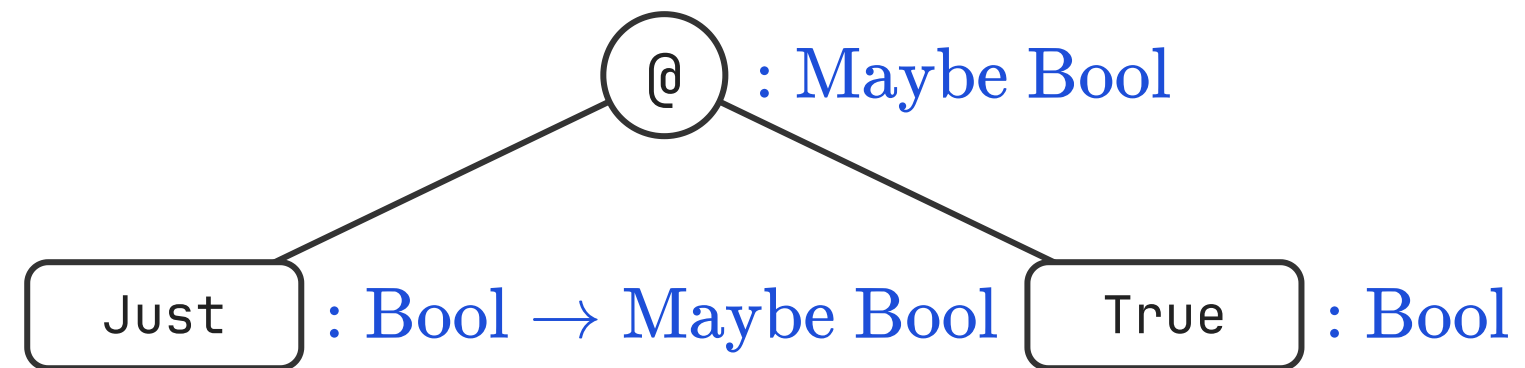
ghci> :kind Int Bool
error: Expected kind '* -> k', but 'Int' has kind '*'
```

- **Δ Кайнды** — система типов для типов: следит, чтобы вычисления на типах были осмысленными и завершались

Три этажа типизации: Just True

Этаж термов: вершинам дерева терма `Just True` приписаны типы — пока как записи.

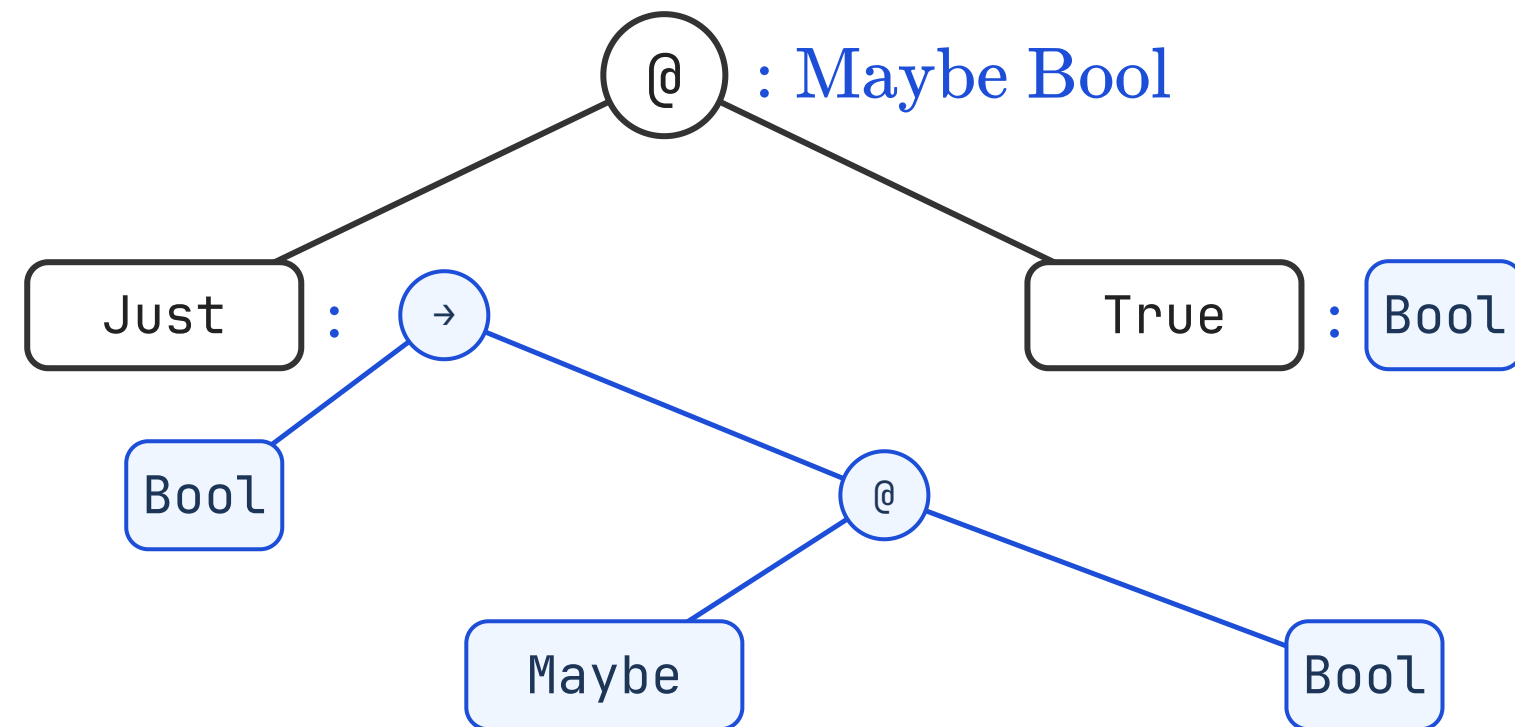
Just True



Три этажа типизации: Just True

Но тип — тоже дерево: у `Just` это стрелка с поддеревьями `Bool` и аппликацией `Maybe` к `Bool`; у `True` — дерево из одного листа.

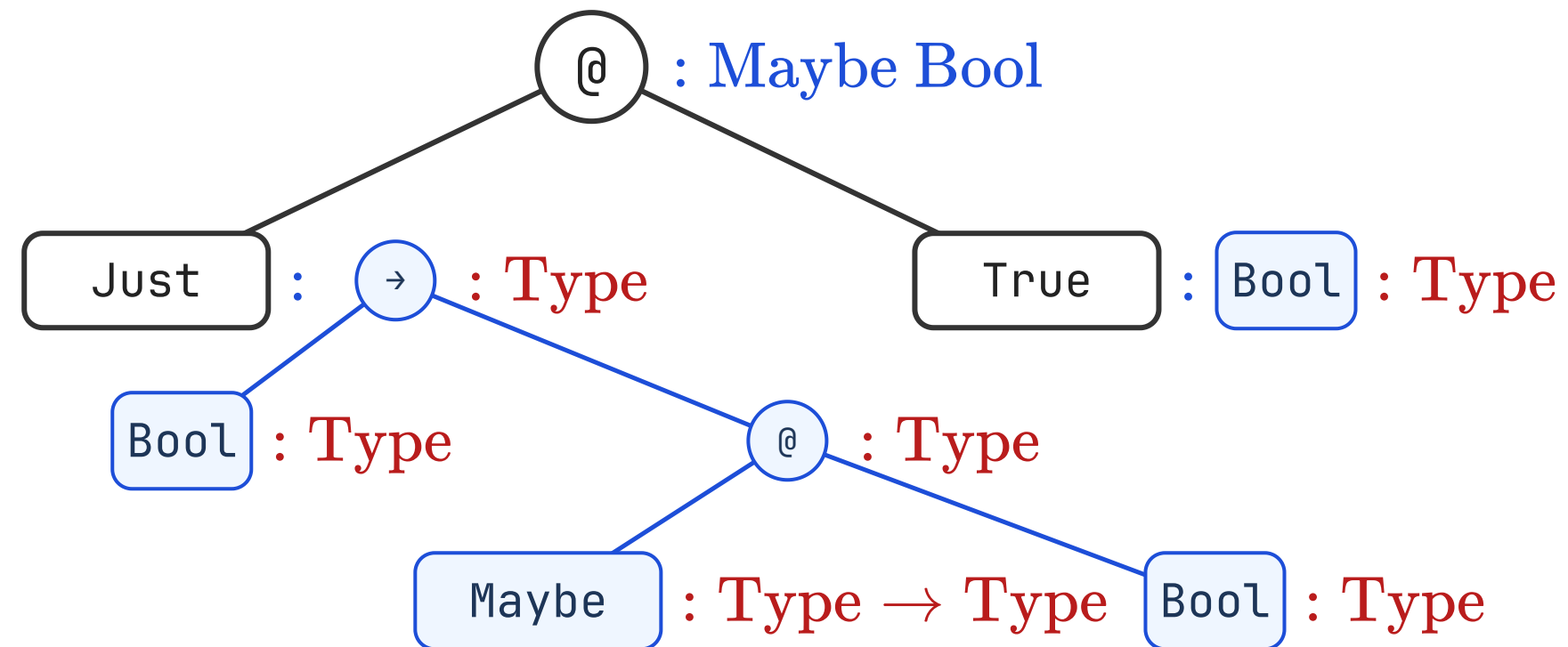
Just True



Три этажа типизации: Just True

Этаж типов: теперь уже вершинам деревьев типов приписаны кайнды. Аппликация типа к типу снимает стрелку в кайнде, как аппликация терма снимает стрелку в типе.

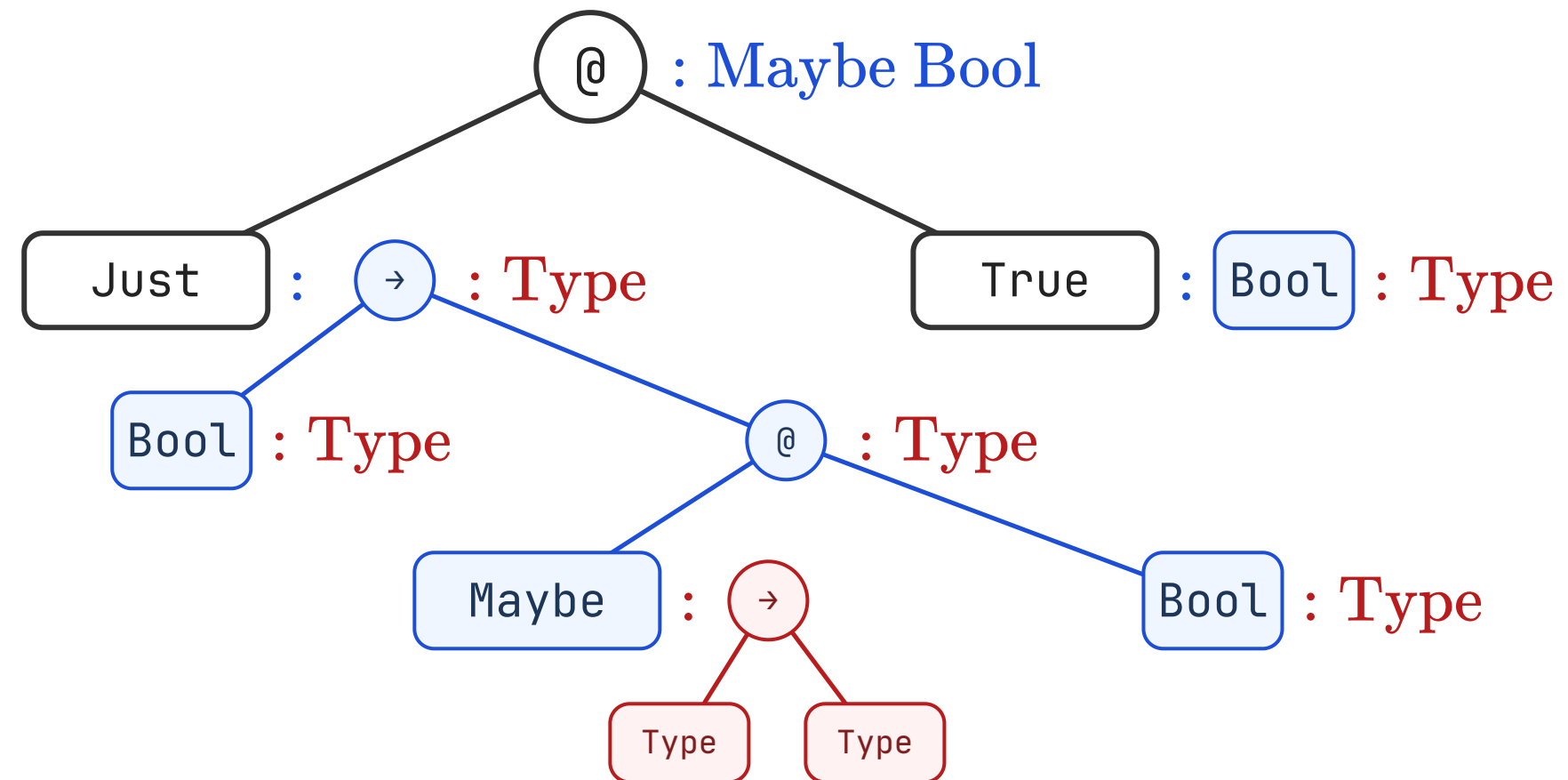
Just True



Три этажа типизации: Just True

И кайнды — тоже деревья: $Type \rightarrow Type$ у *Maybe* — стрелка из двух листьев *Type*, остальные — деревья из одного листа. Что приписывается их вершинам — в конце раздела.

Just True



Правила кайндизации: те же правила, этажом выше

λ-исчисление на типах

$$\frac{\alpha : \kappa_1, \Gamma \vdash \tau : \kappa_2}{\Gamma \vdash \lambda \alpha : \kappa_1. \tau : \kappa_1 \rightarrow \kappa_2} \text{KLam}$$

$$\frac{\Gamma \vdash \tau : \kappa_1 \rightarrow \kappa_2 \quad \Gamma \vdash \sigma : \kappa_1}{\Gamma \vdash \tau \sigma : \kappa_2} \text{KApp}$$

Конструкции обычных типов

$$\frac{\Gamma \vdash \tau : * \quad \Gamma \vdash \sigma : *}{\Gamma \vdash \tau \rightarrow \sigma : *} \text{KArrow}$$

$$\frac{\alpha : \kappa, \Gamma \vdash \tau : *}{\Gamma \vdash \forall \alpha : \kappa. \tau : *} \text{KAll}$$

Пара Чёрча в исчислении

Pair : $* \rightarrow * \rightarrow *$

Pair = $\lambda\tau^* \sigma^*. \forall\gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma$

pair : $\forall\alpha \beta. \alpha \rightarrow \beta \rightarrow \mathbf{Pair} \alpha \beta$

pair = $\Lambda\alpha^* \beta^*. \lambda x^\alpha y^\beta. (\Lambda\gamma^*. \lambda f^{\alpha \rightarrow \beta \rightarrow \gamma}. f \ x \ y)$

fst : $\forall\alpha \beta. \mathbf{Pair} \alpha \beta \rightarrow \alpha$

fst = $\Lambda\alpha^* \beta^*. \lambda p^{\mathbf{Pair} \alpha \beta}. p \ \alpha \ (\mathbf{K} \ \alpha \ \beta)$

fst в действии

Достанем первую компоненту пары. Типы здесь — такие же аргументы, как значения.

```
fst nat bool (pair nat bool 1 true)
```

fst в действии

Раскрыли **fst** и передали типы компонент: $\alpha \mapsto \text{nat}$, $\beta \mapsto \text{bool}$.

```
(λp. p nat (K nat bool)) (pair nat bool 1 true)
```

fst в действии

Пара подставлена вместо p . Дальше она ждёт тип результата и функцию.

`(pair nat bool 1 true) nat (K nat bool)`

fst в действии

Раскрыли **pair**: внутри полиморфная функция, и первым она принимает тип результата γ .

$(\Lambda \gamma. \lambda f. f \ 1 \ \text{true}) \ \text{nat} \ (\text{K nat bool})$

fst в действии

$\gamma \mapsto \text{nat}$: пара узнала, что из неё хотят достать число.

```
(λf. f 1 true) (K nat bool)
```

fst в действии

Функция-выборщик получила обе компоненты.

K nat bool 1 true

fst в действии

К вернула первую. Каждый квантор в типах **fst** и **Pair** — это шаг, на котором кто-то выбрал тип.

1

Задачи: кайнды и пары

✎ Постройте по правилам кайндизации дерево вывода **Pair** : $* \rightarrow * \rightarrow *$. Какие правила и в каком порядке в нём применяются? Поредуцируйте тип **Pair** nat bool до нормальной формы.

✎ Реализуйте на Haskell функции `snd` и `swap :: Pair a b -> Pair b a`. Запишите `snd` в исчислении с явными типовыми абстракциями и аппликациями.

⚡Решение.

Дерево вывода — на следующем слайде. Редукция — два β -шага, как у **Pair** Int Int: нормальная форма $\forall \gamma. (\text{nat} \rightarrow \text{bool} \rightarrow \gamma) \rightarrow \gamma$.

```
snd :: Pair a b -> b
snd p = p (\_ y -> y)

swap :: Pair a b -> Pair b a
swap p = \f -> p (\x y -> f y x)      -- или pair (snd p) (fst p)
```

snd = $\Lambda \alpha^* \beta^*. \lambda p^{\mathbf{Pair} \alpha \beta}. p \beta (\mathbf{K}_* \alpha \beta)$

Дерево вывода: кайнд Pair

$$\Gamma = \tau : *, \sigma : *, \gamma : *$$

$$\begin{array}{c}
\frac{}{\Gamma \vdash \tau : *} \text{ctx} \quad \frac{}{\Gamma \vdash \sigma : *} \text{ctx} \quad \frac{}{\Gamma \vdash \gamma : *} \text{ctx} \\
\frac{}{\Gamma \vdash \tau : *} \text{ctx} \quad \frac{\Gamma \vdash \sigma : * \quad \Gamma \vdash \gamma : *}{\Gamma \vdash \sigma \rightarrow \gamma : *} \text{KArrow} \\
\frac{\Gamma \vdash \tau : * \quad \Gamma \vdash \sigma \rightarrow \gamma : *}{\Gamma \vdash \tau \rightarrow \sigma \rightarrow \gamma : *} \text{KArrow} \\
\frac{\Gamma \vdash \tau \rightarrow \sigma \rightarrow \gamma : * \quad \Gamma \vdash \gamma : *}{\Gamma \vdash (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma : *} \text{KArrow} \\
\frac{\Gamma \vdash (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma : *}{\tau : *, \sigma : * \vdash \forall \gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma : *} \text{KAll} \\
\frac{\tau : *, \sigma : * \vdash \forall \gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma : *}{\tau : * \vdash \lambda \sigma. \forall \gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma : * \rightarrow *} \text{KLam} \\
\frac{\tau : * \vdash \lambda \sigma. \forall \gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma : * \rightarrow *}{\vdash \lambda \tau \sigma. \forall \gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma : * \rightarrow * \rightarrow *} \text{KLam}
\end{array}$$

ШАГИ

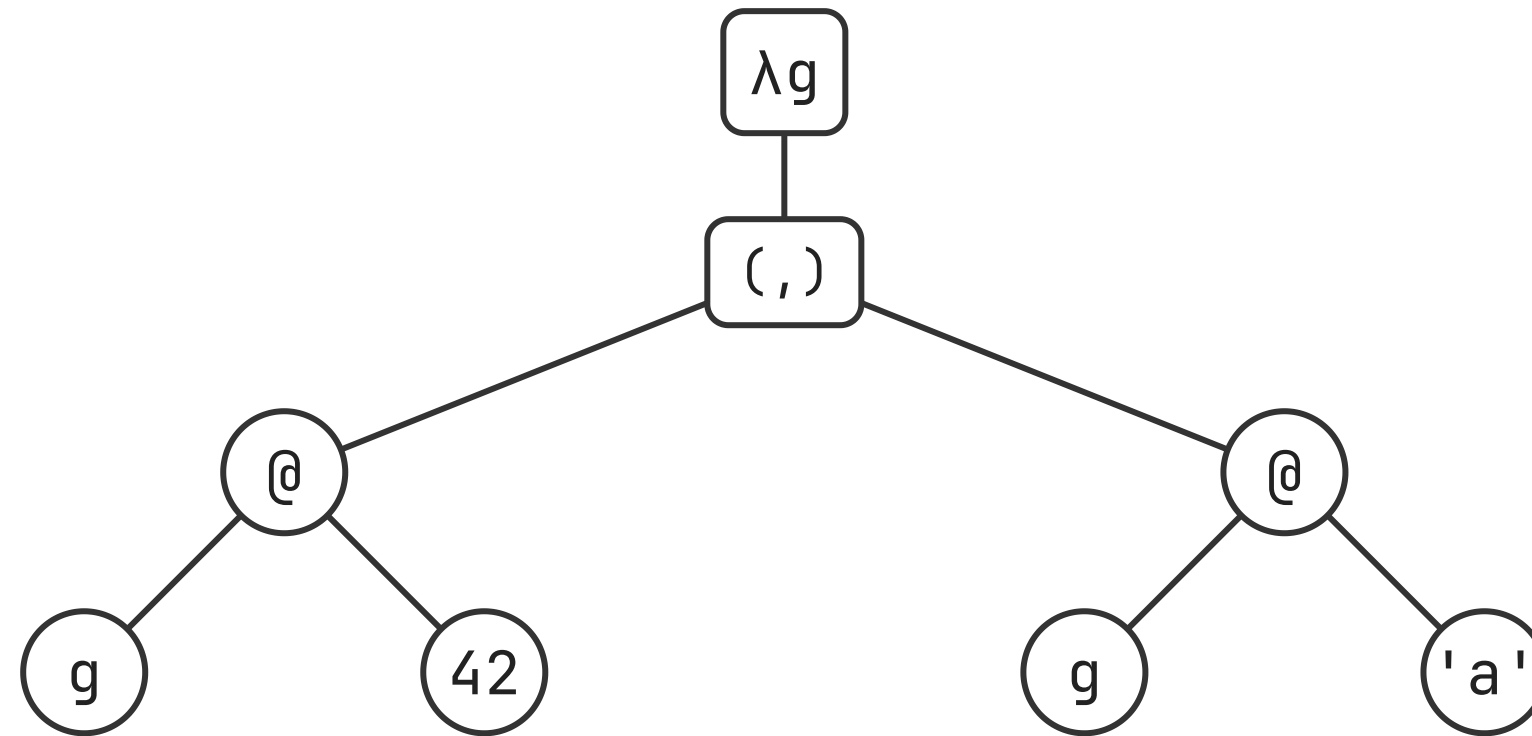
1. KLam : снимаем $\lambda\tau$
2. KLam : снимаем $\lambda\sigma$
3. KAll : снимаем $\forall\gamma$
4. KArrow : внешняя стрелка
5. KArrow : $\tau \rightarrow (\sigma \rightarrow \gamma)$; $\text{ctx}: \gamma$
6. $\text{ctx}: \tau$; KArrow : $\sigma \rightarrow \gamma$
7. $\text{ctx}: \sigma$ и γ

First-class polymorphism

Вывод типов спотыкается

Программа корректна: передай в неё `id` — получишь пару. Выведем её тип, как это делает алгоритм Хиндли–Милнера.

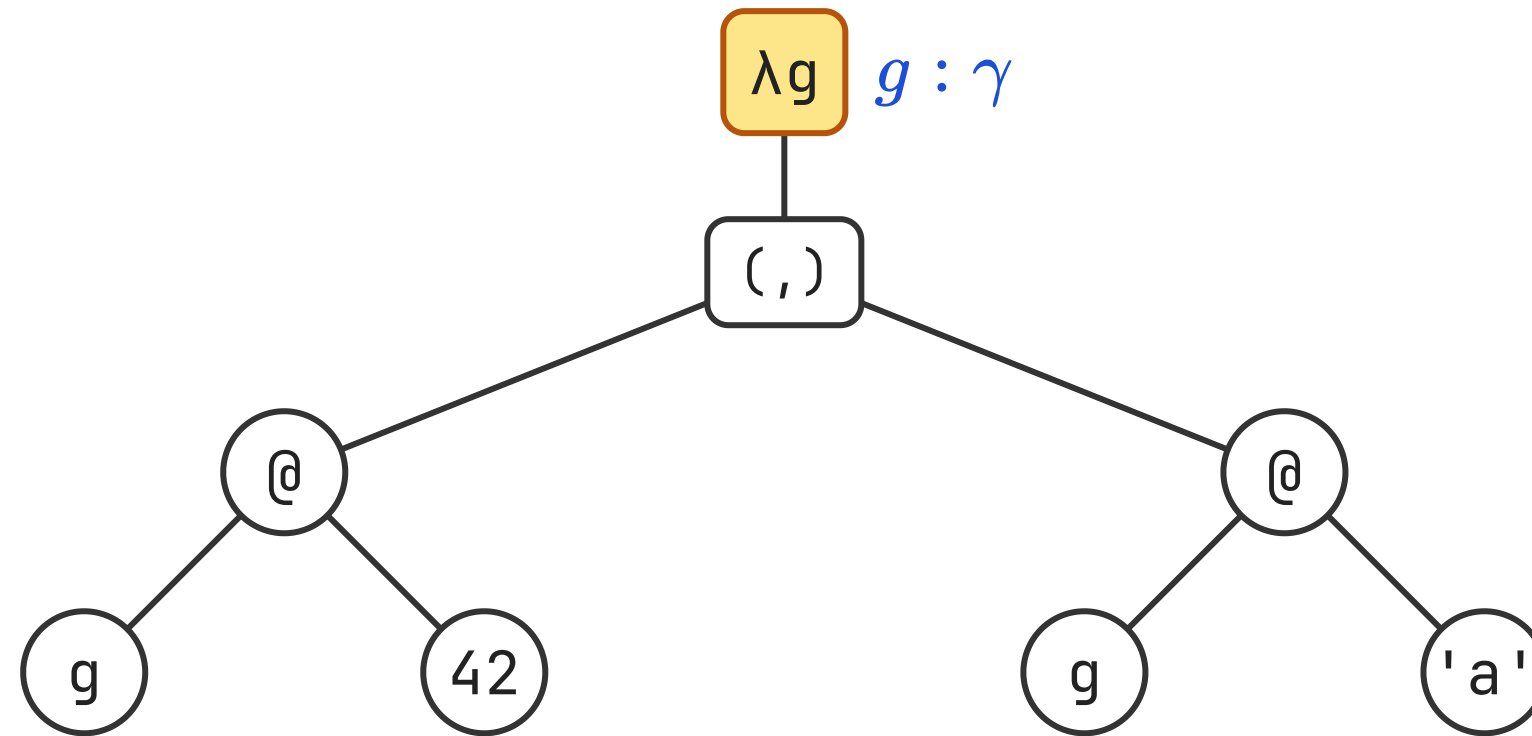
$\lambda g. (g\ 42, g\ 'a')$



Вывод типов спотыкается

Аргументу функции алгоритм приписывает **один** тип γ , без кванторов внутри.

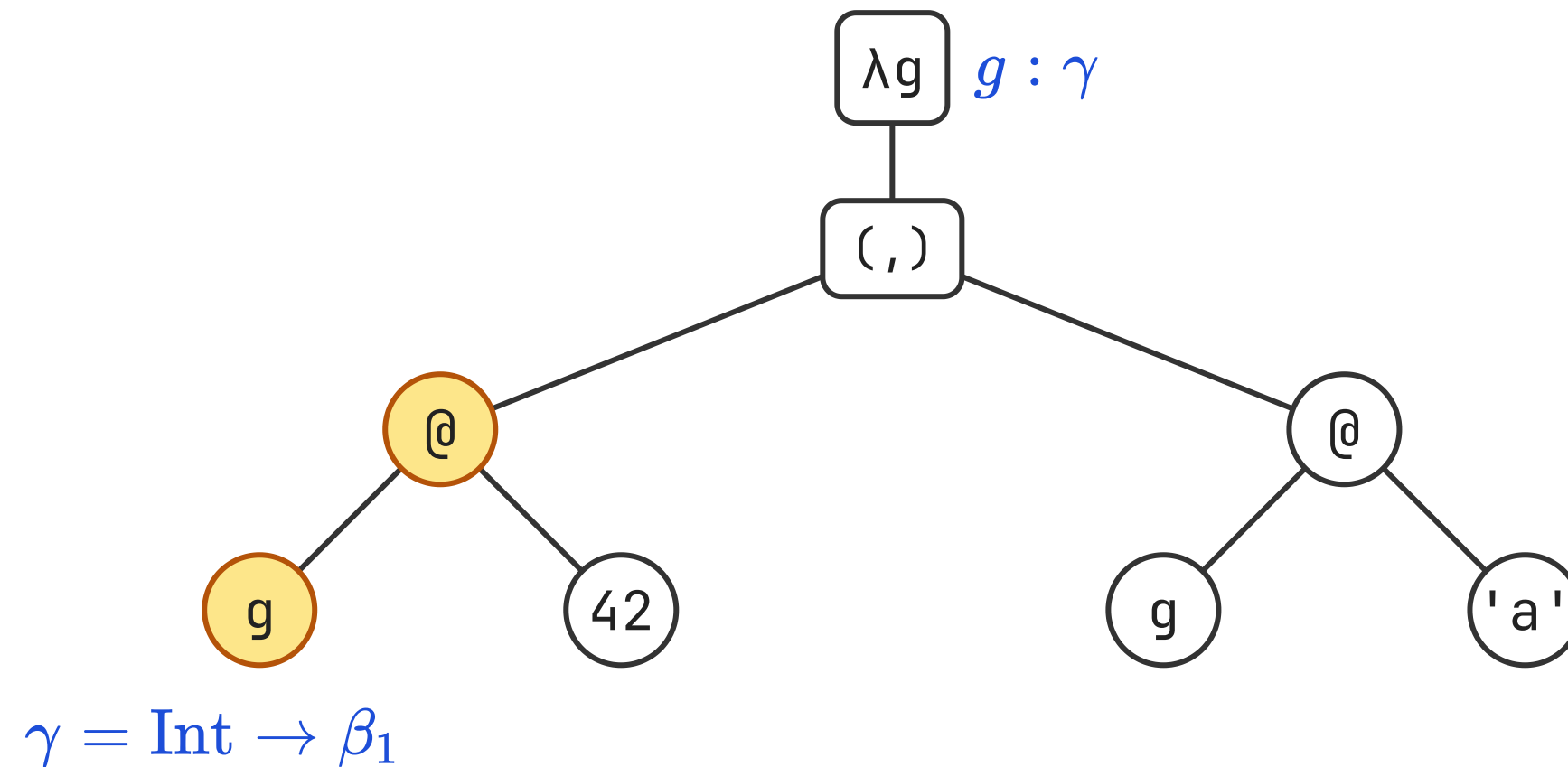
$\lambda g. (g\ 42, g\ 'a')$



Вывод типов спотыкается

Первое применение: g применяют к числу, значит $\gamma = \text{Int} \rightarrow \beta_1$.

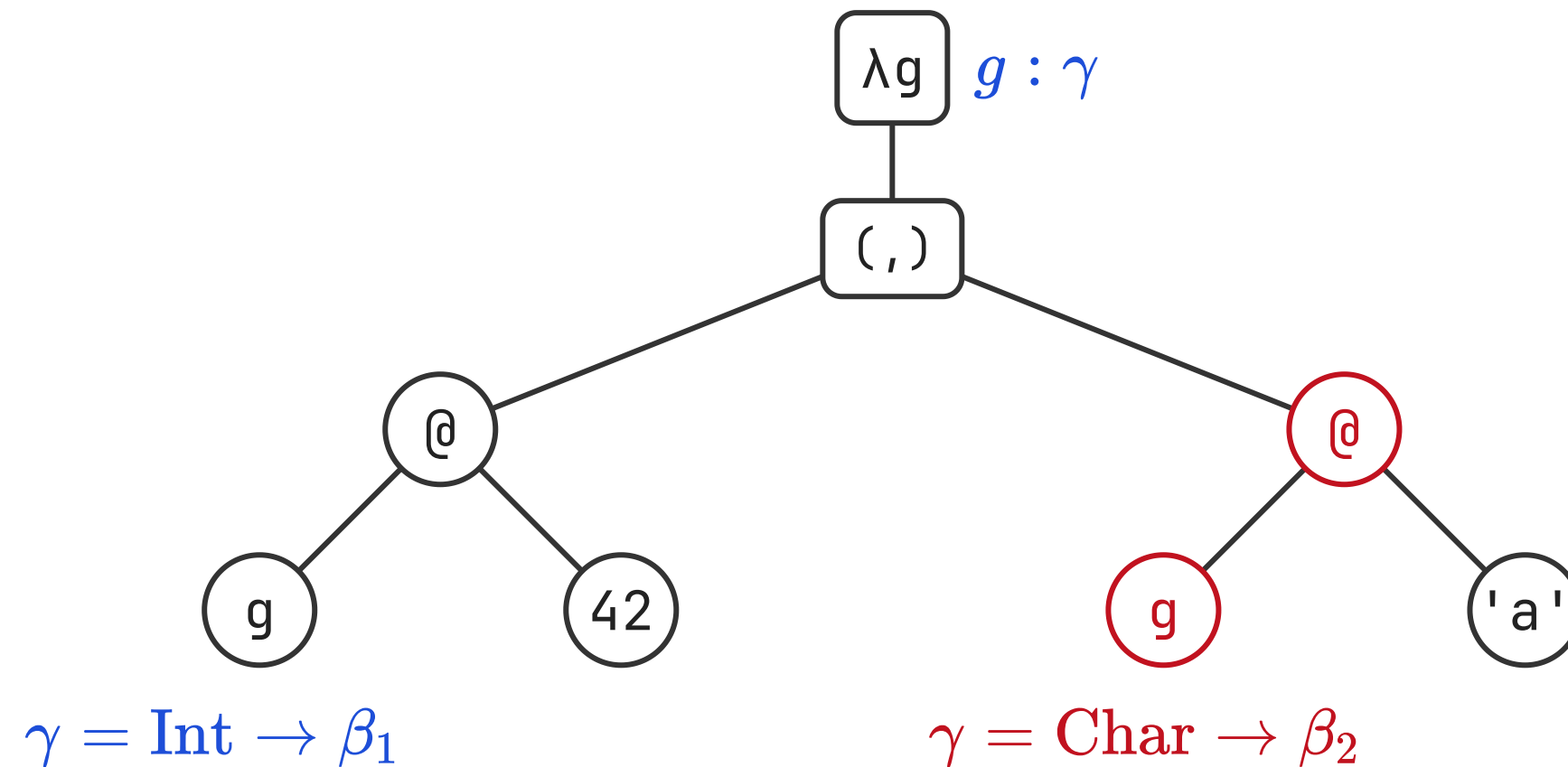
$\lambda g. (g\ 42, g\ 'a')$



Вывод типов спотыкается

Второе: $\gamma = \text{Char} \rightarrow \beta_2$. Но γ уже занята, а $\text{Int} \neq \text{Char}$ — корректная программа отвергнута.

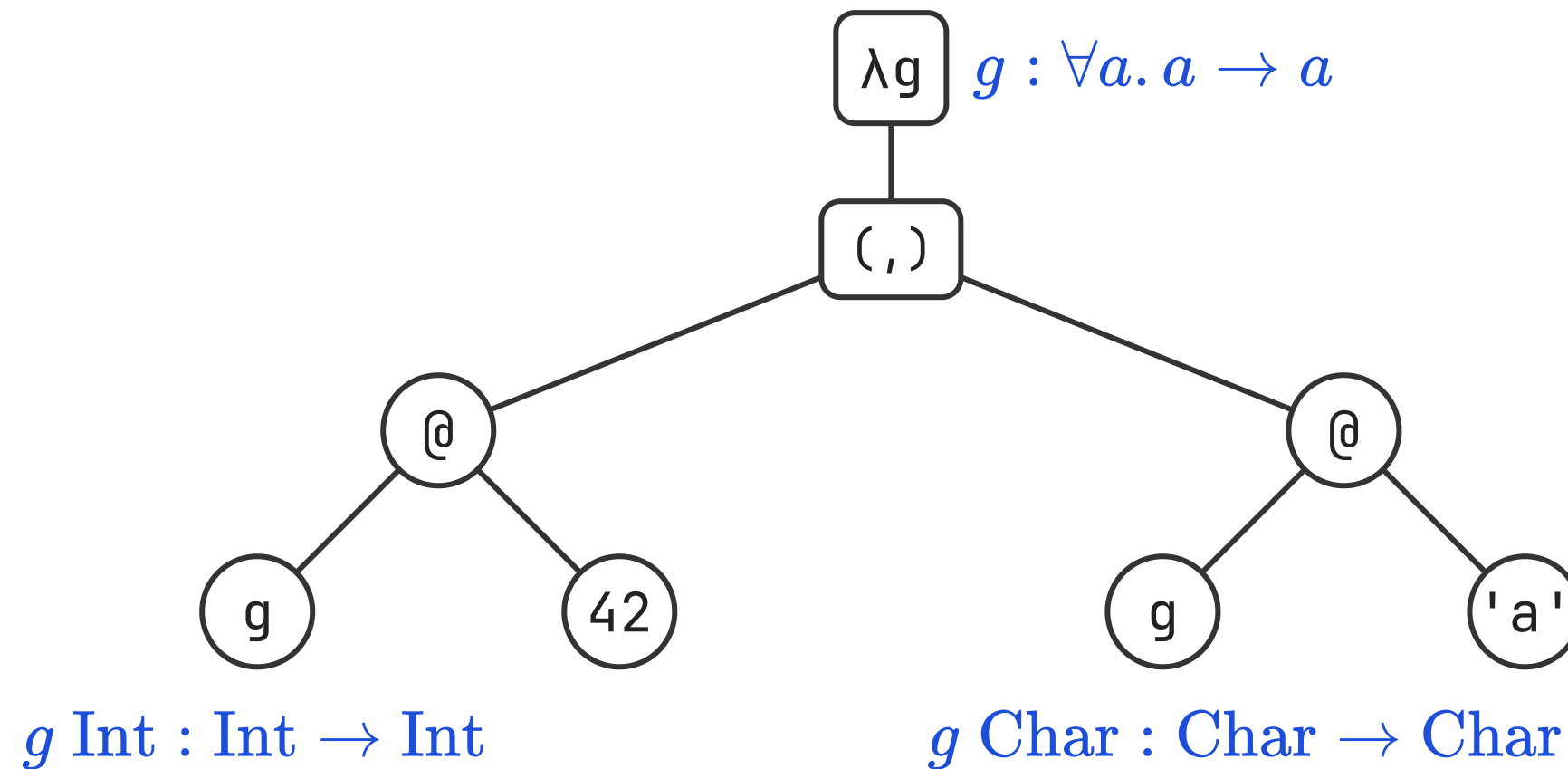
$\lambda g. (g\ 42, g\ 'a')$



Вывод типов спотыкается

Лекарство — квантор **слева от стрелки**: тип для g выбирает сама функция, в каждом месте свой. Это тип высшего ранга.

$\lambda g. (g\ 42, g\ 'a')$



Типы высшего ранга в Haskell

```
f :: (forall a . a -> a) -> (Int, Char)      -- RankNTypes
f g = (g @Int 42, g @Char 'a')
```

```
ghci> f (\x -> x)
(42, 'a')
```

- — Цена: типы высшего ранга не выводятся — их приписывают руками, остальное выводится как раньше

 **Устно:** Какую ошибку выдаст GHC, если убрать у `f` сигнатуру и типовые аппликации? Какой тип для `g` он пытался вывести?

 **Устно:** Какой ранг имеет тип `Int -> (forall a . a -> a)`?

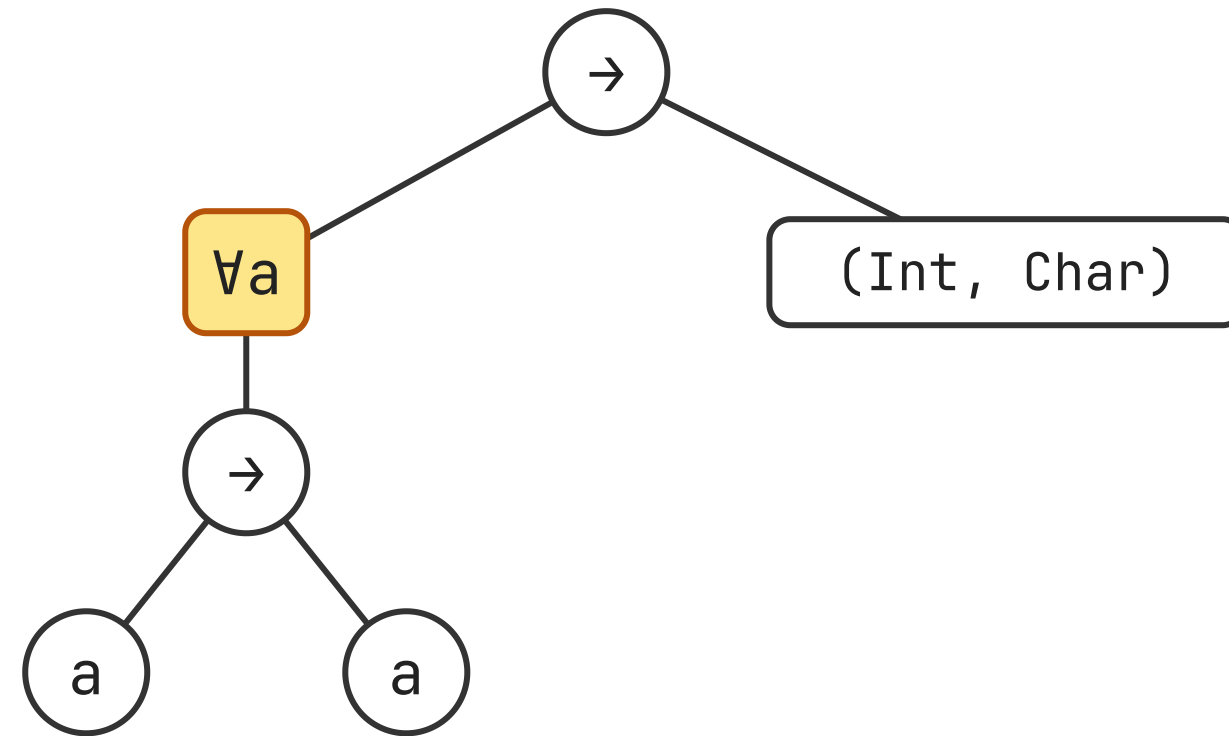
 **Решение.**

Couldn't match expected type 'Int' with actual type 'Char': первое применение зафиксировало `g :: Int -> b`. Про ранг — на следующем слайде.

Ранг: где стоит квантор

Тип — дерево. Ранг считает, сколько раз по пути к квантору мы свернули **налево** от стрелки. Здесь один раз: ранг 2.

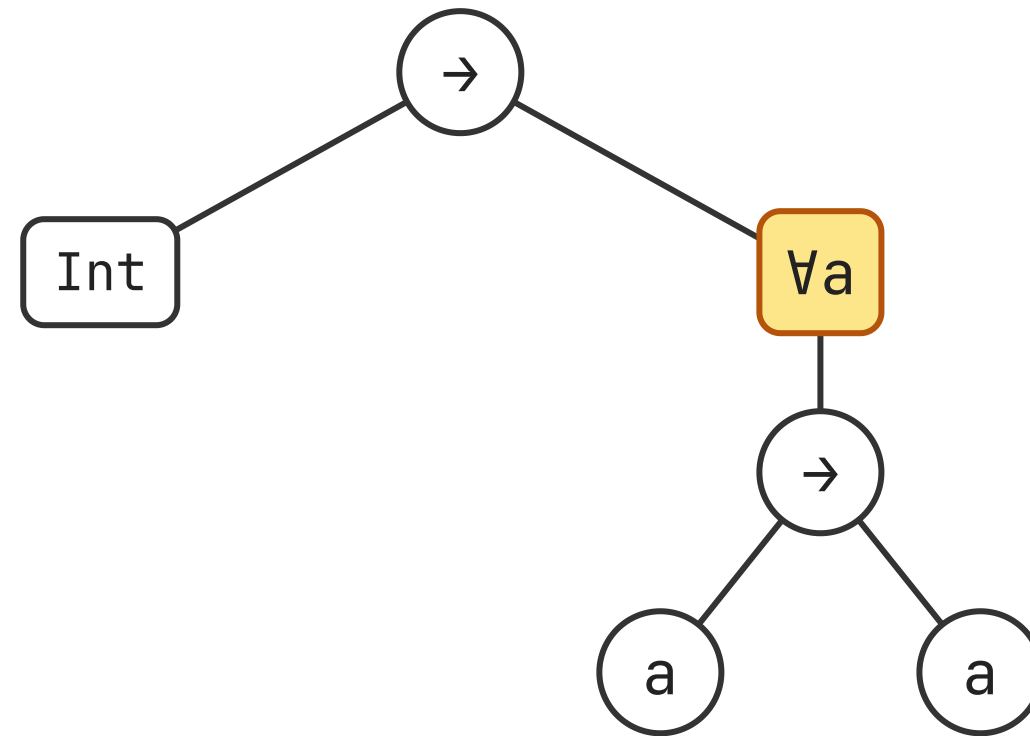
$(\forall a. a \rightarrow a) \rightarrow (\text{Int}, \text{Char})$



Ранг: где стоит квантор

А в этом типе квантор стоит **справа** от стрелки...

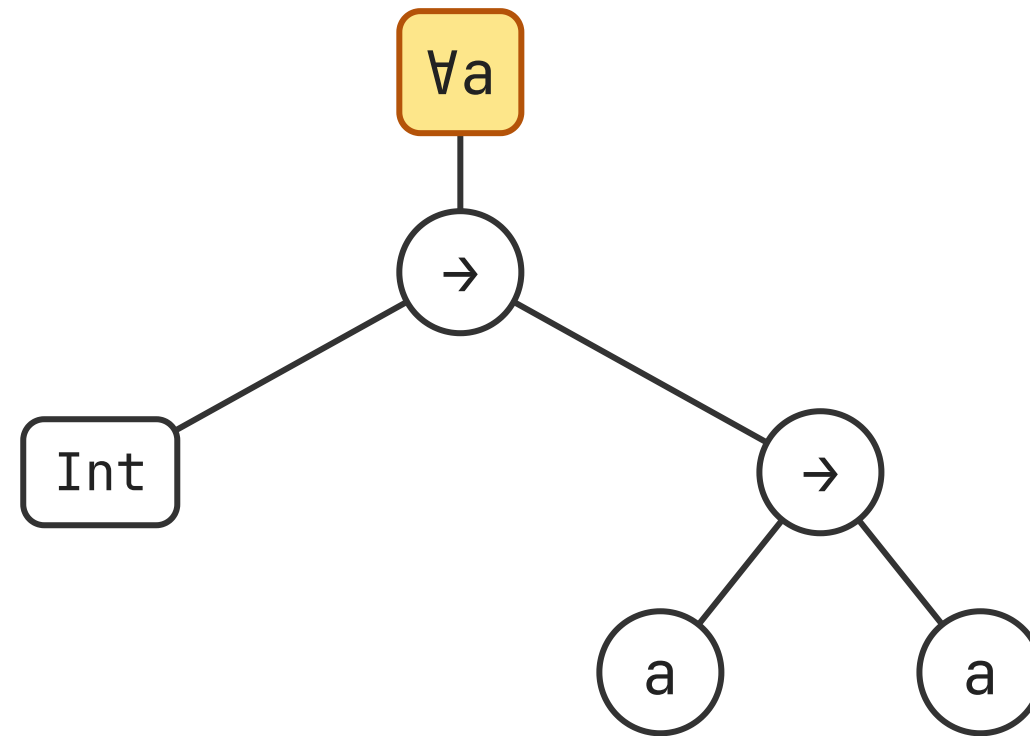
$\text{Int} \rightarrow (\forall a. a \rightarrow a)$



Ранг: где стоит квантор

...и его можно вынести наружу, ничего не изменив. Поворотов налево нет: ранг 1.

$\forall a. \text{Int} \rightarrow a \rightarrow a$



Числа Чёрча и обёртки

```
suc :: (forall a . (a -> a) -> a -> a) -> (b -> b) -> b -> b    -- ранг 2
suc n s z = s (n s z)
```

```
newtype Church = Church (forall a . (a -> a) -> a -> a)           -- квантор под конструктором
(+) :: Church -> Church -> Church                                   -- ранг 1
```

Та же обёртка на Kotlin

```
interface Church {  
    fun <a> fold(s: (a) -> a, z: a): a           // полиморфный метод: квантор под интерфейсом  
}  
  
fun plus(n: Church, m: Church): Church = object : Church {  
    override fun <a> fold(s: (a) -> a, z: a): a = n.fold(s, m.fold(s, z))  
}
```

Импредикативное применение

Оператор $\$$ — обычная функция. Чему равна типовая переменная a в выражении `runST $ do ...`?

$$(\$) :: (a \rightarrow b) \rightarrow a \rightarrow b$$
$$\text{runST} :: (\forall s. \text{ST } s \text{ } c) \rightarrow c$$

Импредикативное применение

a специализируется на **полиморфный** тип. По умолчанию так нельзя; для **\$** в GHC сделано исключение, для остальных функций — `ImpredicativeTypes`.

$$(\$) :: ((\forall s. ST\ s\ c) \rightarrow c) \rightarrow (\forall s. ST\ s\ c) \rightarrow c$$
$$\text{runST} :: (\forall s. ST\ s\ c) \rightarrow c$$

ST-trick: мутабельные ячейки в ограниченной области

```
runST :: (forall s. ST s a) -> a

sumTo :: Int -> Int
sumTo n = runST do
  ref <- newSTRef 0
  forM [0..n] \i -> modifySTRef ref (+ i)
  readSTRef ref
```

```
newSTRef :: v -> ST s (STRef s v)
```

```
ghci> runST (newSTRef True) -- попробуем вынести ссылку наружу
error: Couldn't match type 'a' with 'STRef s Bool'
       because type variable 's' would escape its scope
```

s не может сбежать

Квантор по s стоит слева от стрелки: его область действия — только аргумент **runST**.

$\text{runST} :: (\forall s. \text{ST } s \ a) \rightarrow a$

s не может сбежать

Вернуть ссылку — значит взять $a = \text{STRef } s \text{ Bool}$. Но тогда s в результате оказывается **вне** области квантора: такого типа не существует.

$\text{runST} :: (\forall s. \text{ST } s (\text{STRef } s \text{ Bool})) \rightarrow \text{STRef } s \text{ Bool}$

Higher-kinded polymorphism

Абстракция по контейнеру

Один `fmap` для списков, `Maybe` и деревьев: переменная f стоит на месте **конструктора типов**.

$$\text{fmap} :: (a \rightarrow b) \rightarrow \boxed{f} \ a \rightarrow \boxed{f} \ b$$
$$\boxed{f} :: \text{Type} \rightarrow \text{Type}$$

Абстракция по контейнеру

Подставляем список...

$$\text{fmap} :: (a \rightarrow b) \rightarrow [] a \rightarrow [] b$$
$$[] :: \text{Type} \rightarrow \text{Type}$$

Абстракция по контейнеру

...Maybe...

$\text{fmap} :: (a \rightarrow b) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b$

$\text{Maybe} :: \text{Type} \rightarrow \text{Type}$

Абстракция по контейнеру

...или дерево. В Haskell абстрагироваться можно по типам любых кайндов — и в функциях, и в `data`.

`fmap :: (a → b) → Tree a → Tree b`

`Tree :: Type → Type`

Параметр — конструктор типов

```
newtype Fix f = Fix (f (Fix f))                -- Fix :: (Type -> Type) -> Type
cata :: forall (f :: Type -> Type) a . Functor f => (f a -> a) -> Fix f -> a
```

✎ Какой кайнд у `Fix Maybe`? Приведите несколько значений этого типа. Какой знакомый тип они напоминают?

⚡ **Решение.**

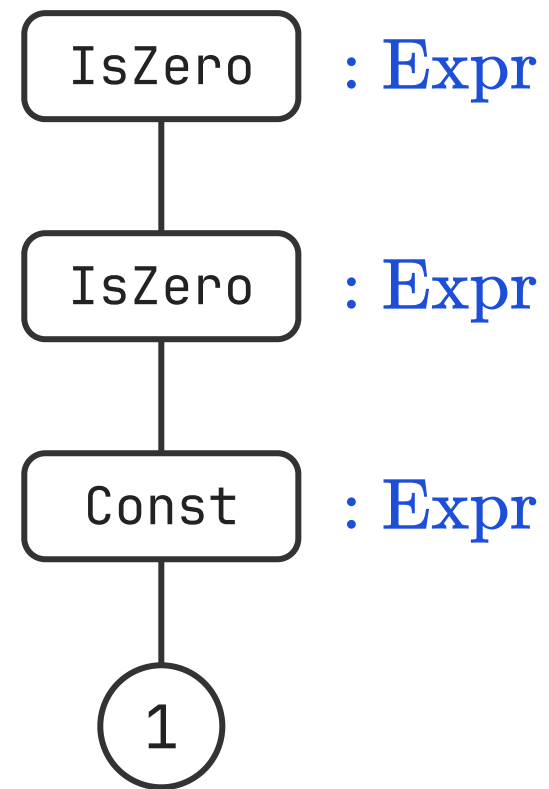
`Fix Maybe :: Type`; значения — `Fix Nothing`, `Fix (Just (Fix Nothing))`, `Fix (Just (Fix (Just (Fix Nothing))))`, ... — натуральные числа Пеано.

GADTs

Некорректная программа — корректное значение

`data Expr = Const Int | IsZero Expr | If Expr Expr Expr`. Типу `Expr` всё равно, что внутри: каждый узел — `Expr`, значение Haskell корректно. А программа бессмысленна.

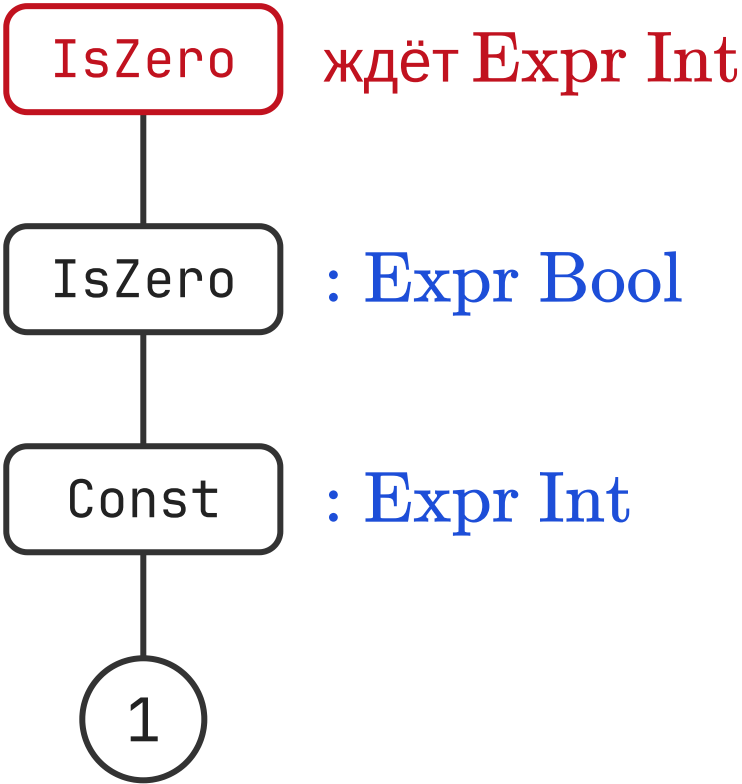
`IsZero (IsZero (Const 1))`



Некорректная программа — корректное значение

Добавим типу индекс — тип, в который выражение вычисляется. Теперь **IsZero** ждёт число, а получает булево: ошибка видна в типах.

IsZero (IsZero (Const 1))



Обобщённые алгебраические типы данных

```
data Expr ty where                                -- data через типы конструкторов
  Const :: Int -> Expr Int
  IsZero :: Expr Int -> Expr Bool
  If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty
```

```
eval :: Expr ty -> ty
eval = \case
  Const x    -> x                -- ty ~ Int
  IsZero t   -> eval t == 0      -- ty ~ Bool
  If c t e   -> if eval c then eval t else eval e
```

- — Цена: при матчинге по GADT типы без подсказки не выводятся — сигнатура `eval` обязательна

Задача: пары в языке

✎ Добавьте в язык пары: конструкторы `MkPair :: Expr a -> Expr b -> Expr (a, b)` и `Fst :: Expr (a, b) -> Expr a`. Допишите `eval`. Какие равенства типов возникают в новых ветках?

⚡Решение.

```
MkPair :: Expr a -> Expr b -> Expr (a, b)
Fst    :: Expr (a, b) -> Expr a

eval = \case
  ...
  MkPair a b -> (eval a, eval b)    -- ty ~ (a, b)
  Fst p -> fst (eval p)             -- ty ~ a; b — экзистенциальный, виден только внутри ветки
```

Структуры на уровне типов, data promotion

Вектор с длиной в типе

```
data Zero                                -- натуральные числа на уровне типов, по Пеано
data Suc n

data Vec (size :: Type) (elem :: Type) where
  VNil  :: Vec Zero a
  VCons :: a -> Vec n a -> Vec (Suc n) a
```

 **Устно:** Сколько обитателей у типа `Suc (Suc Zero)`?

 **Решение.**

Ни одного (кроме $\perp$): типы пустые, они нужны только как индексы.

Длина растёт вместе с вектором

Пустой вектор: длина — ноль.

$VNil$

$:: Vec\ Zero\ a$

Длина растёт вместе с вектором

Каждый **VCons** добавляет элемент в значение и **Suc** в тип.

VCons 2 VNil
:: Vec (**Suc** Zero) Int

Длина растёт вместе с вектором

Длина вектора известна компилятору.

VCons 1 (VCons 2 VNil)

:: Vec (**Suc** (Suc Zero)) Int

Безопасный zip

```
vzip :: Vec n a -> Vec n b -> Vec n (a, b)
vzip VNil VNil = VNil -- n ~ Zero
vzip (VCons x xs) (VCons y ys) = VCons (x, y) (vzip xs ys) -- n ~ Suc n'
```

```
vzip VNil (VCons y ys) = VNil -- а что с векторами разной длины?
```

```
error: Couldn't match type 'Zero' with 'Suc n1'
      Inaccessible code in a pattern with constructor VCons
```

Задача: добавление в конец

✎ Напишите функцию добавления элемента в конец вектора. Двигайтесь последовательно, заполняя типовые дыры и отслеживая возникающие эквивалентности.

⚡ Решение.

```
snoc :: Vec n a -> a -> Vec (Suc n) a
snoc VNil y = VCons y VNil           -- n ~ Zero: нужен Vec (Suc Zero) a
snoc (VCons x xs) y = VCons x (snoc xs y) -- n ~ Suc m: нужен Vec (Suc (Suc m)) a
```

Кайнды слишком бедны

```
ghci> :kind Vec (Maybe Int) Bool      -- вектор длины «Maybe Int»?
Vec (Maybe Int) Bool :: Type          -- принято
```

- — `Suc :: Type -> Type`, и индексом вектора может оказаться любой тип

Свой кайнд для индекса

```
type data Nat = Zero | Suc Nat           -- TypeData: новый кайнд и его типы

data Vec (size :: Nat) (elem :: Type) where
  VNil  :: Vec Zero a
  VCons :: a -> Vec n a -> Vec (Suc n) a
```

```
ghci> :kind Vec (Maybe Int) Bool
error: Expected kind 'Nat', but 'Maybe Int' has kind 'Type'
```

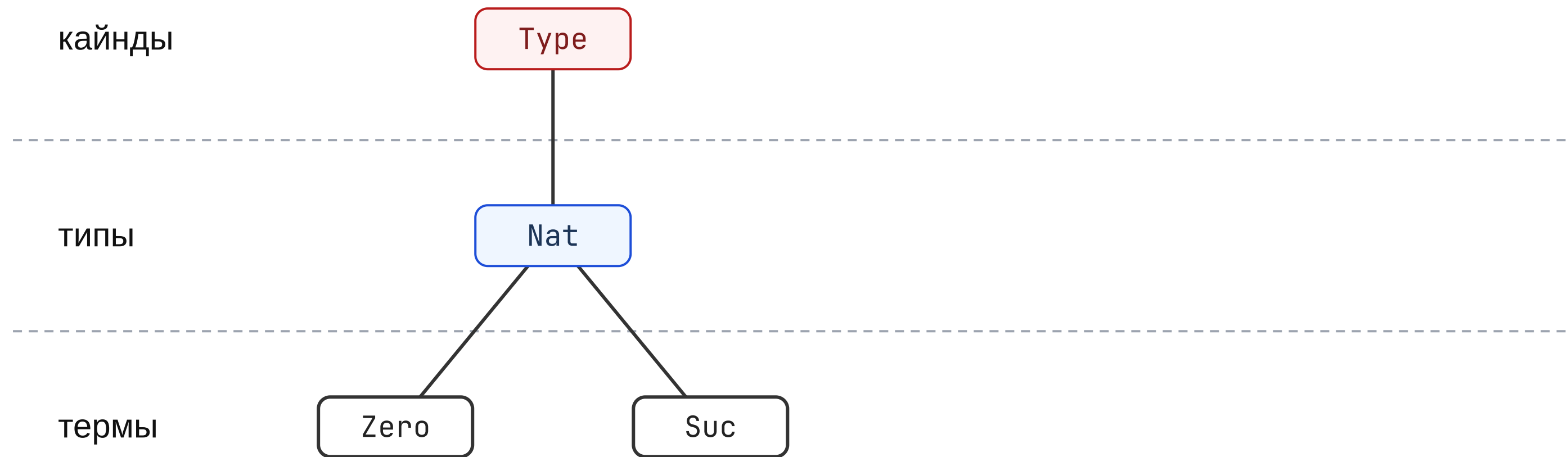
 **Устно:** Что выведет `ghci> :k Vec`?

 **Решение.**

```
Vec :: Nat -> Type -> Type
```

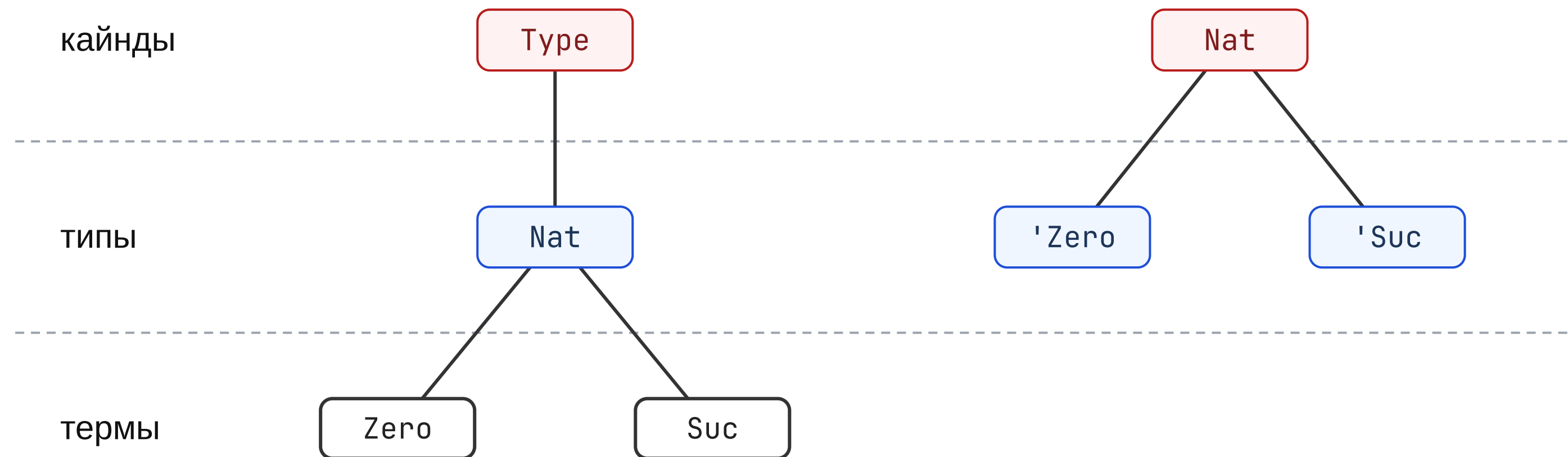
Продвижение: на этаж выше

Обычное объявление `data Nat = Zero | Suc Nat`: тип `Nat` и его значения.



Продвижение: на этаж выше

DataKinds продвигает объявление на этаж выше: конструктор типа становится кайндом, конструкторы данных — типами.



Продвижение в GHCi

```
data Nat = Zero | Suc Nat
```

```
ghci> :k Suc          -- Suc здесь используется как тип
Suc :: Nat -> Nat
```

```
data T = T Nat          -- конструктор типа и конструктор данных — тѣзки
```

```
ghci> :k T
T :: Type              -- про конструктор типа
ghci> :k 'T
'T :: Nat -> T         -- про продвинутый конструктор данных
```

```
data [a] = [] | (:) a [a] -- полиморфный тип даёт полиморфный кайнд (PolyKinds)
```

```
ghci> :k '(:)
'(:) :: forall k . k -> [k] -> [k]
```

Три этажа: примеры продвижений

Term	Type	Kind
Zero	Nat	Type
[Zero, Suc Zero]	[Nat]	Type
[]	forall a. [a]	Type
(:)	forall a. a -> [a] -> [a]	Type
	'Suc 'Zero	Nat
	'['Zero, 'Suc 'Zero]	[Nat]
	'[Int, Double]	[Type]
	'[]	forall k. [k]
	'(:)	forall k. k -> [k] -> [k]

Индексы любых кайндов

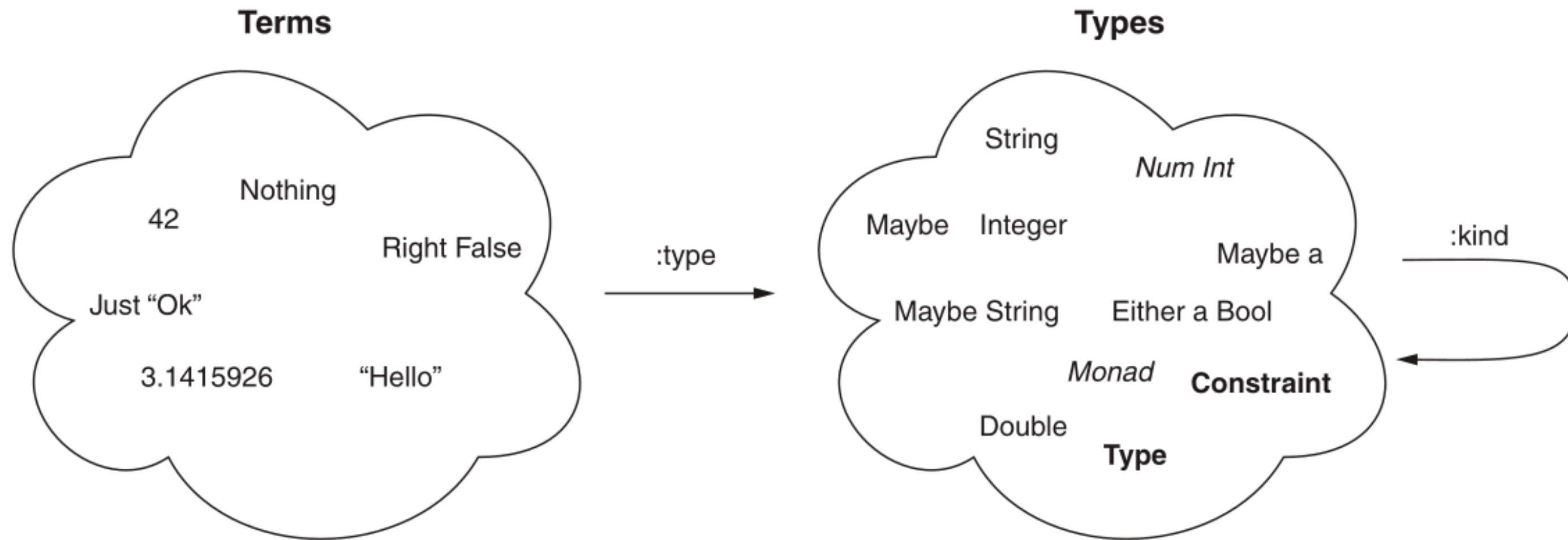
```
data HList (tys :: [Type]) where                -- индекс — список типов
  HNil :: HList '[]
  HCons :: ty -> HList tys -> HList (ty ': tys)

example :: HList '[Int, Bool, Double]
example = HCons 42 $ HCons True $ HCons 12.5 HNil
```

```
newtype Tagged (tag :: k) (a :: Type) = Tagged a -- тег любого кайнда

example :: Tagged ("dbId" :: Symbol) Int        -- например, строка уровня типов
example = Tagged 42
```

Типы и кайнды — одно

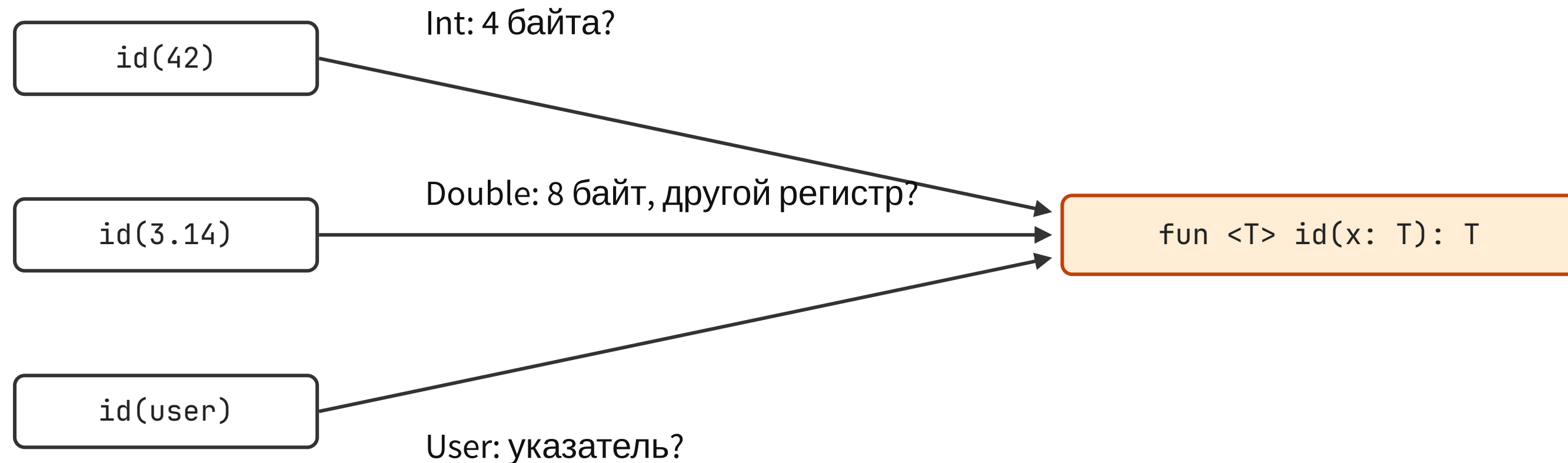


```
ghci> :kind Type
Type :: Type
```

Реализация параметрического полиморфизма

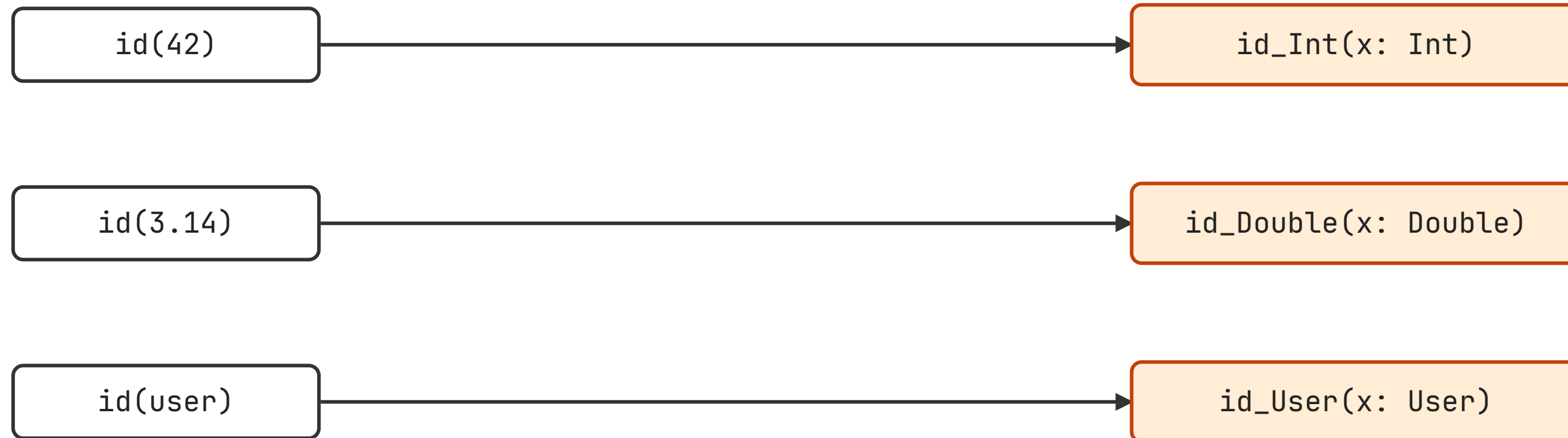
Одна функция — четыре способа её скомпилировать

Конвенция вызова: где лежит аргумент — регистр, стек, куча — и какого он размера. Она зависит от типа, а тип **T** неизвестен.



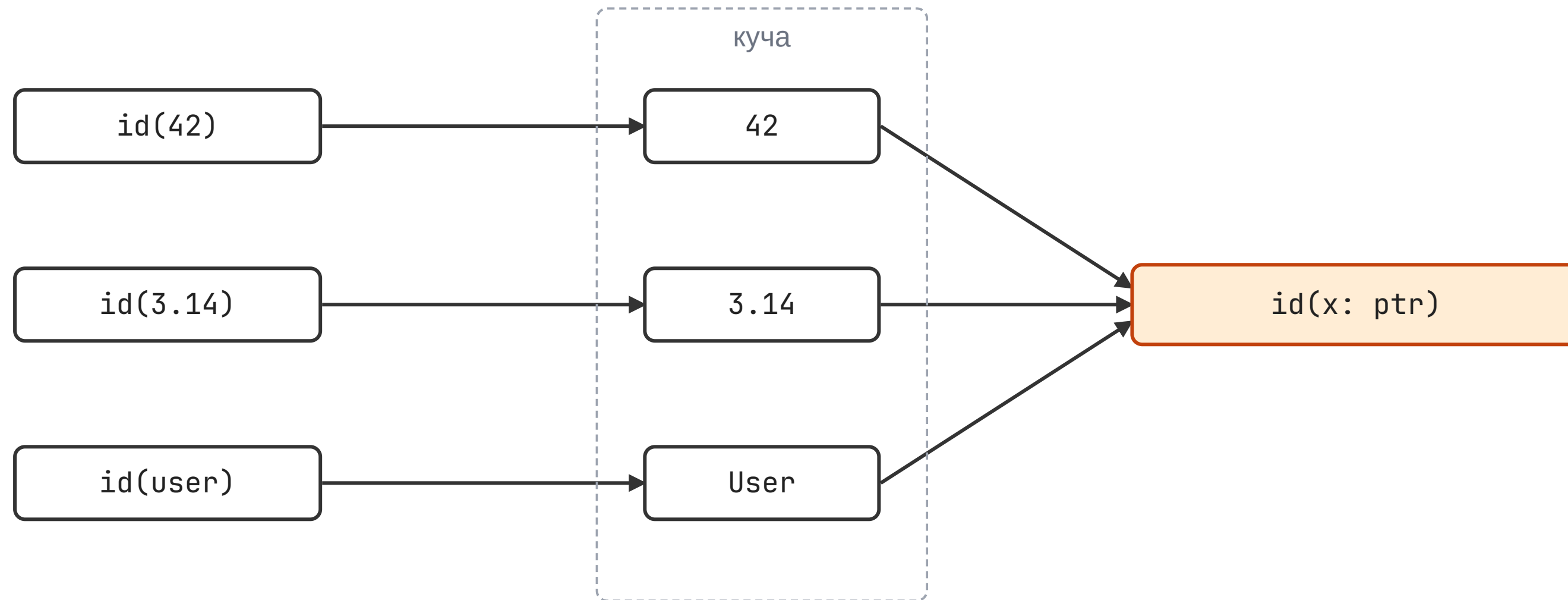
Одна функция — четыре способа её скомпилировать

Мономорфизация (C++, Rust): копия кода на каждый набор типов. Самый быстрый код; долгая компиляция, большой бинарник, нет полиморфных методов в интерфейсах.



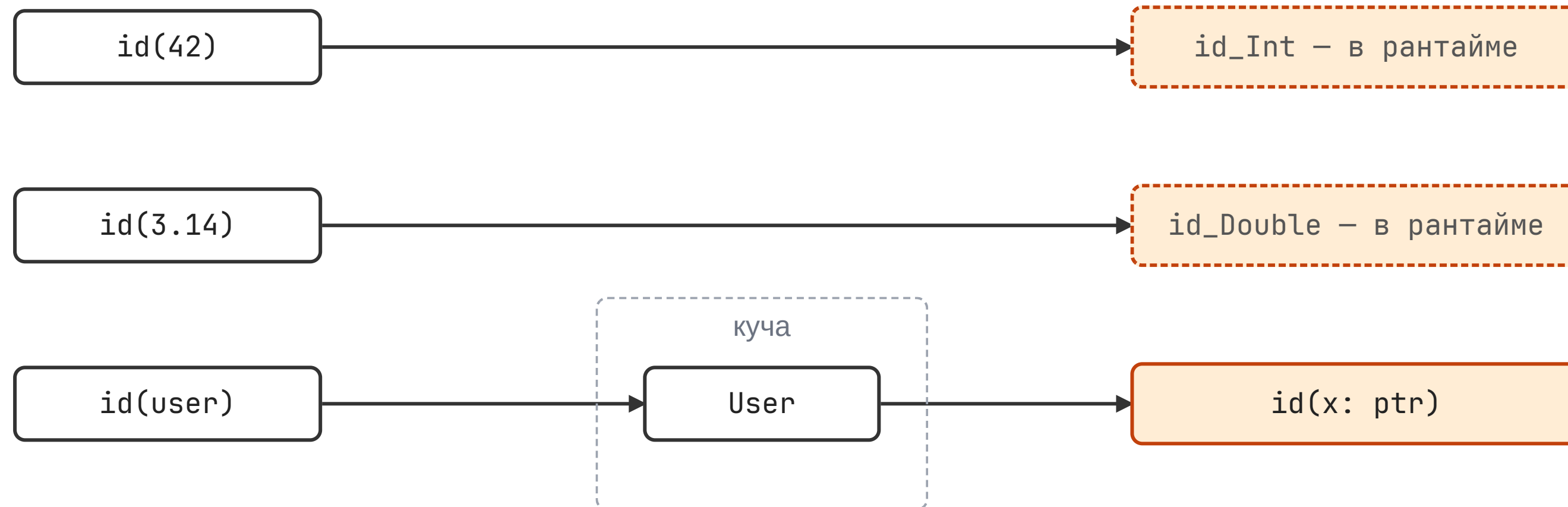
Одна функция — четыре способа её скомпилировать

Стирание типа (JVM, Haskell): значения — в кучу, код один и ждёт указатели. Быстрая компиляция и гибкость; платим боксингом и индирекциями.



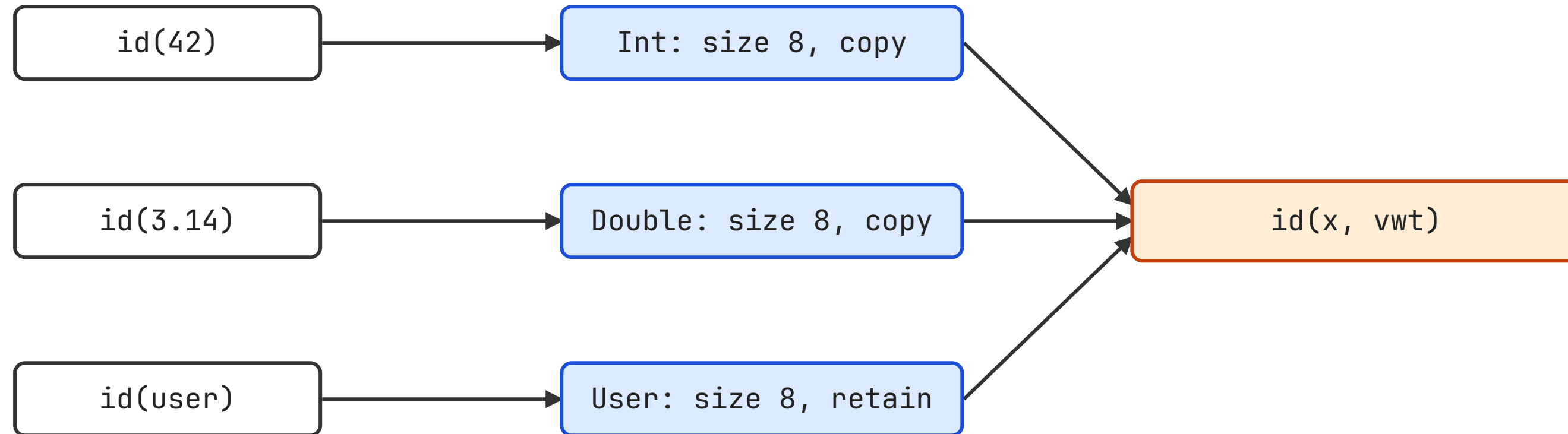
Одна функция — четыре способа её скомпилировать

Гибрид (C#): для ссылочных типов один код с указателями, для value-типов специализации появятся лениво, в рантайме.



Одна функция — четыре способа её скомпилировать

Таблица свойств типа (Swift): код один, а размер и способ копирования приходят вместе с типом — в таблице. Платим вызовами через таблицу.



Цена каждого подхода

	мономорфизация	стирание	гибрид	таблица свойств
языки	C++, Rust	JVM, Haskell	C#	Swift
скорость кода	+ лучшая	- боксинг	+ value-типы	- вызовы через таблицу
время компиляции, размер	-	+	+	+
динамическая загрузка	-	+	+	+
полиморфные методы, higher-rank	-	+	+	+
тип известен в рантайме	- только статически	- стёрт	+	+

Таблица свойств типа в Swift

```
struct value_witness_table {  
    size_t size, align;  
    void (*copy_init)(opaque *dst, const opaque *src, type *T);  
    void (*copy_assign)(opaque *dst, const opaque *src, type *T);  
    void (*move_init)(opaque *dst, opaque *src, type *T);  
    void (*move_assign)(opaque *dst, opaque *src, type *T);  
    void (*destroy)(opaque *val, type *T);  
};
```

Example:

```
func f<T>(_ t: T) -> T {  
    let copy = t  
    return copy  
}
```

Implementation:

```
void f(opaque *result, opaque *t, type *T) {  
    opaque *copy = alloca(T->vwt->size);  
    T->vwt->copy_init(copy, t, T);  
    T->vwt->move_init(result, copy, T);  
    T->vwt->destroy(t, T);  
}
```

Задачи: подходы в языках

✎ **Устно:** В Kotlin (JVM, стирание типов) проверка `x is T` для типового параметра `T` разрешена только внутри `inline`-функций, где параметр помечен `reified`. Объясните ограничение. Какой из подходов здесь локально применяется?

✎ **Устно:** В Rust (мономорфизация) трейт с обобщённым методом нельзя использовать как `trait object` (`dyn Trait`). Каким недостатком мономорфизации это объясняется?

⚡ **Решение.**

- Kotlin: в рантайме `T` стёрт; `inline` вставляет тело в место вызова, где `T` известен, — локальная мономорфизация
- Rust: через `vtable` в месте вызова неизвестно, какую специализацию метода вызывать — «в интерфейсах не может быть полиморфных методов»

Задача: плагин

✎ Приложение подгружает плагин, скомпилированный отдельно, и хочет вызвать из него обобщённую функцию с типом, о котором автор плагина не знал. Для каждого из четырёх подходов определите, возможно ли это, и почему.

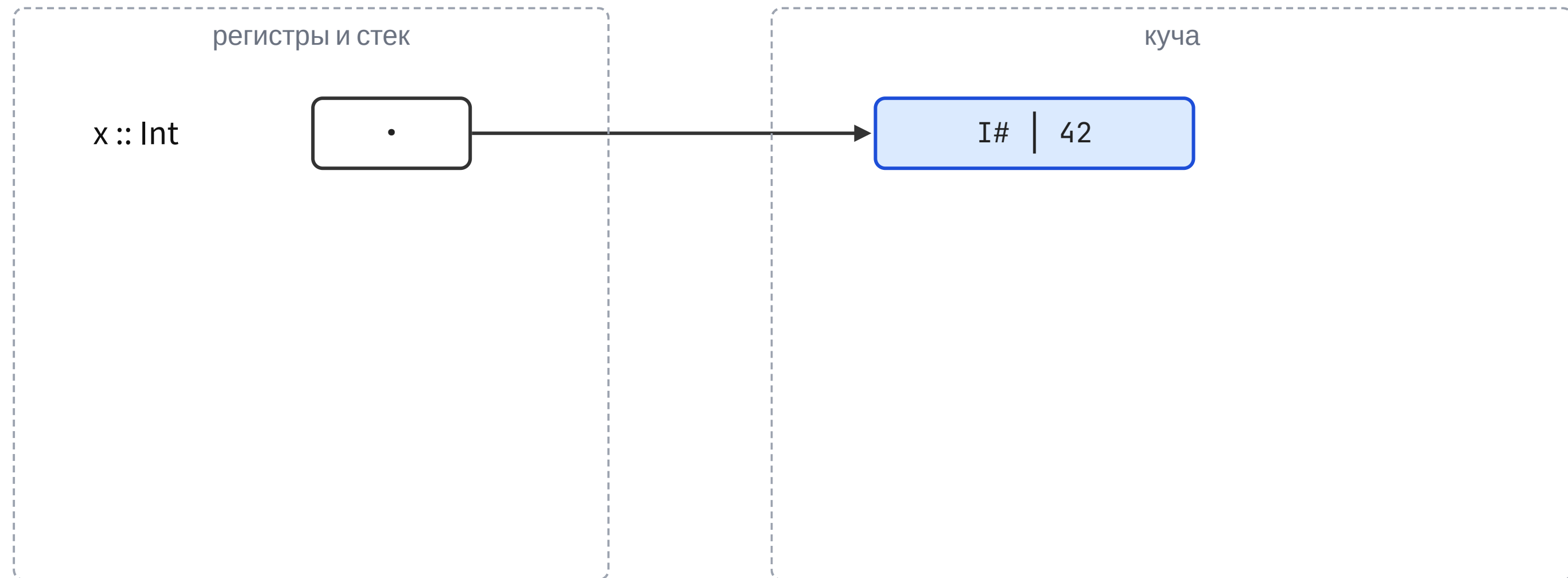
⚡ *Решение.*

- Мономорфизация — нет: кода до специализации не существует, линковаться не к чему
- Стирание — да: один код на все типы, плагин ждёт указатели
- Гибрид — да: для reference-типов готовая специализация, для value-типов рантайм породит её при вызове
- Witness table — да: таблицу свойств типа передаст вызывающий

Полиморфизм по конвенции вызова

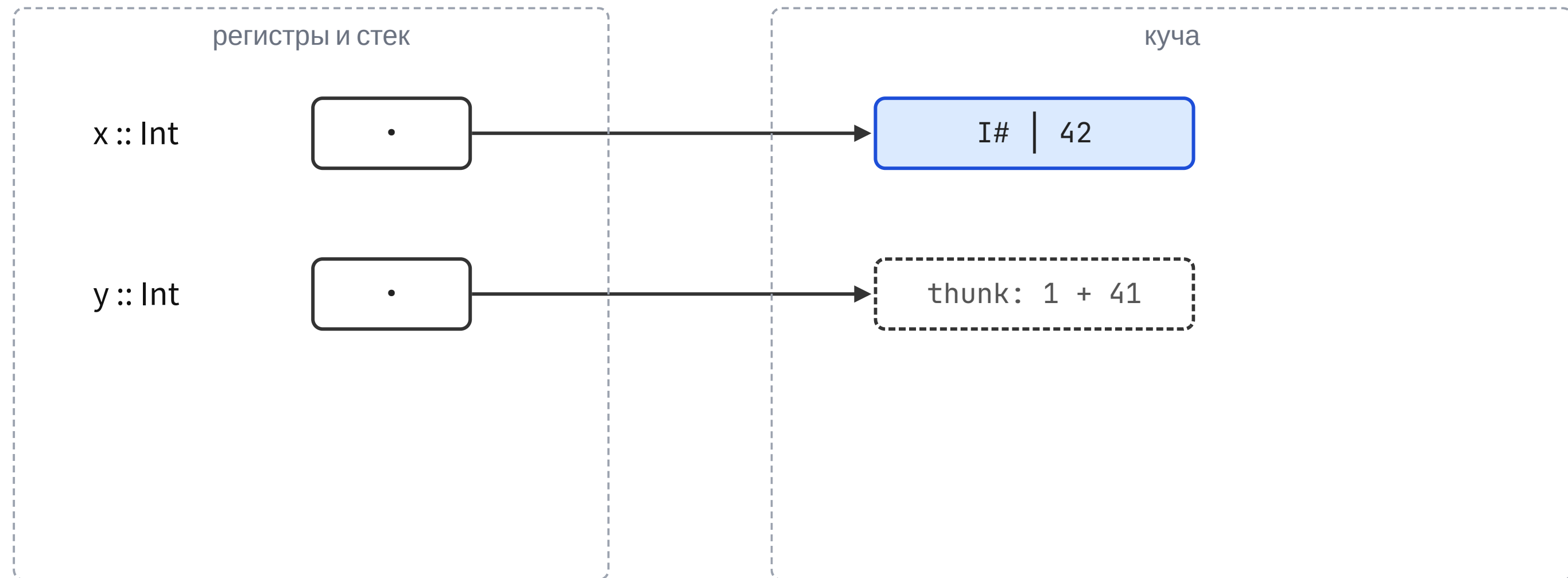
Где лежит значение

$x :: \text{Int}$ — boxed: в переменной указатель, само число в куче. Обычный Int — это $\text{data Int} = \text{I\# Int\#}$.



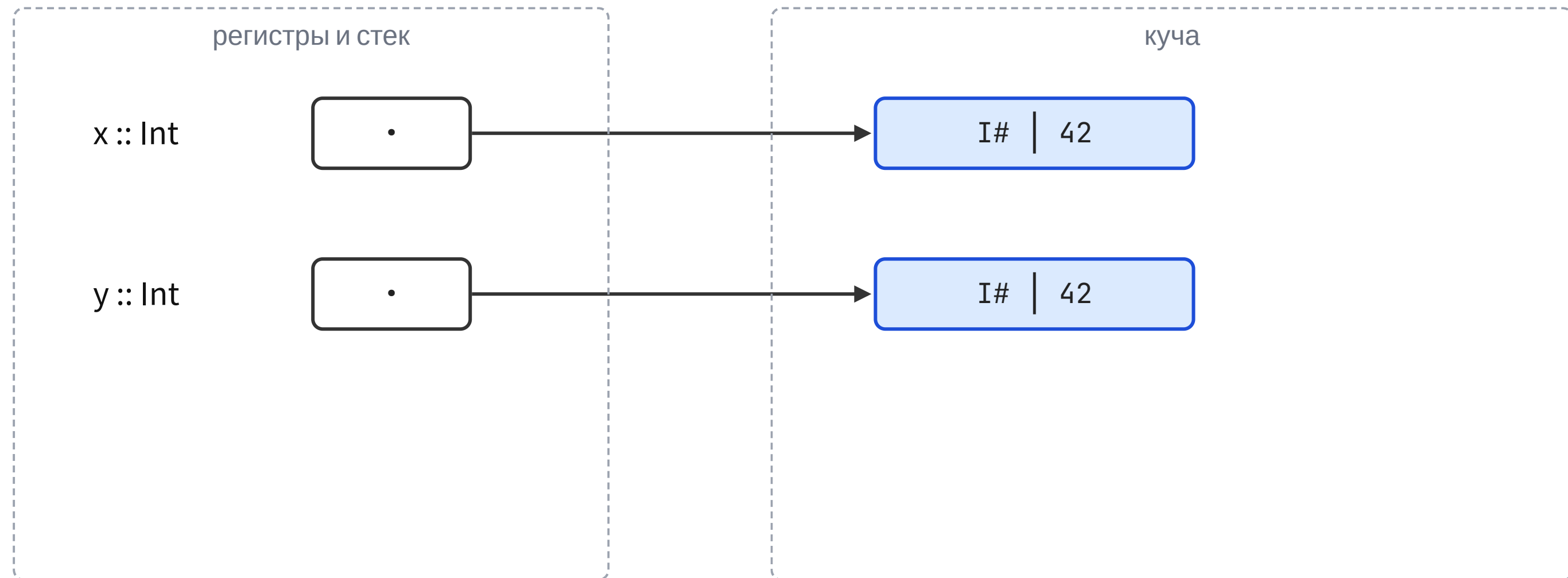
Где лежит значение

И lifted: по указателю может лежать отложенное вычисление, ещё не число.



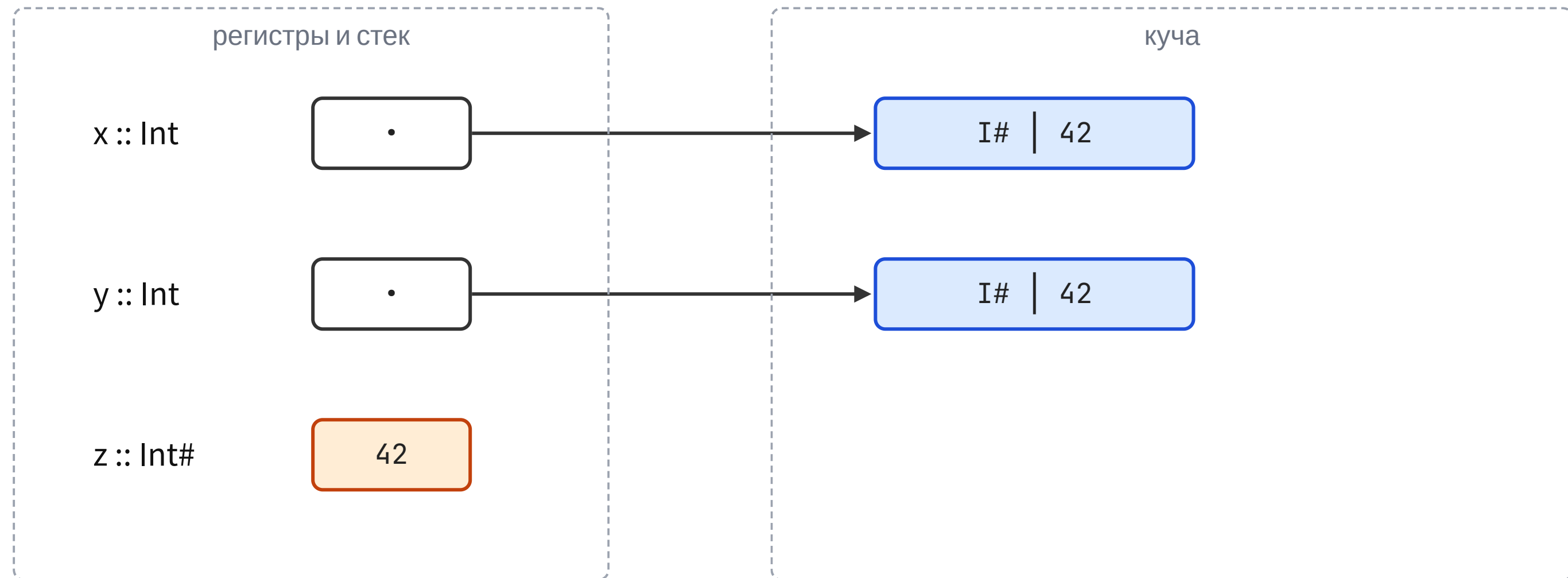
Где лежит значение

Когда значение понадобится, вычисление заменится результатом. За лень и единообразие платим индирекцией и аллокацией.



Где лежит значение

$z :: \text{Int}\#$ — unboxed и unlifted: число прямо в регистре, без кучи и без лени.



Две оси: boxed и lifted

	Boxed	Unboxed
Lifted	<i>Int</i> <i>Bool</i>	
Unlifted	<i>ByteArray</i> _#	<i>Int</i> _# <i>Char</i> _#

- **Boxed** — по указателю, в куче; **unboxed** — по значению
- **Lifted** — содержит $\perp$, может быть отложенным; **unlifted** — всегда вычислен
- Свойство lifted/unlifted — **levity**

```
divMod# :: Int -> Int -> (# Int, Int #)      -- unboxed кортеж: два результата в двух регистрах
case divMod# n k of (# quot, rem #) -> ...
```

Представление — в кайнде

```
TYPE :: RuntimeRep -> Type

data Levity = Lifted | Unlifted

data RuntimeRep = BoxedRep Levity
                | IntRep | DoubleRep
                | TupleRep [RuntimeRep]
                | SumRep [RuntimeRep]
                | ...

type LiftedRep = BoxedRep Lifted

type Type = TYPE LiftedRep
```

Какой кайнд?

- `Int :: TYPE (BoxedRep Lifted)`, или просто `Type`
- `Int# :: TYPE IntRep`
- `Maybe Int :: Type`
- `Maybe :: Type -> Type`
- `(# Int, Bool #) :: TYPE (TupleRep '[LiftedRep, LiftedRep])`
- `(# Int#, (# Int, Double# #) #) :: TYPE (TupleRep '[IntRep, TupleRep '[LiftedRep, DoubleRep]])`

 Какой кайнд у типа `(# Int#, Bool #)`? Проверьте себя в GHCi, включив расширения `MagicHash` и `UnboxedTuples`.

⚡Решение.

```
(# Int#, Bool #) :: TYPE (TupleRep '[IntRep, LiftedRep])
```

Representation polymorphism

```
ghci> :k (->)
(->) :: forall {q :: RuntimeRep} {r :: RuntimeRep}. TYPE q -> TYPE r -> Type
```

 **Устно:** Подумайте, почему функция имеет boxed тип. Может ли быть иначе? Может ли это быть полезным?

Связыватель без представления

```
bad :: forall r (a :: TYPE r). a -> a
bad x = x                                -- куда смотреть за x: регистр? какой? куча?

error: The binder 'x' does not have a fixed runtime representation
```

```
($) :: forall ra rb (a :: TYPE ra) (b :: TYPE rb). (a -> b) -> a -> b
($) f = f                                -- связывателя нет — нет и вопроса
```

Задача: `error`

✎ Функция `error` полиморфна по представлению результата: `error :: forall (r :: RuntimeRep) (a :: TYPE r). String -> a`. Почему это не нарушает ограничение на связыватели?

⚡ Решение.

`a` стоит только в **результате**: ни один параметр `error` не имеет тип `a`, связывателя полиморфного представления нет. А результат никогда не строится — функция бросает исключение, так что и код под произвольное представление генерировать не нужно.

Что мы узнали

System F	скрытый аргумент — тип; Λ и универсальная аппликация, $TLam/TApp$; в Haskell — @
кайнды	λ -исчисление на уровне типов, $KLam/KApp$; три этажа: терм — тип — кайнд, и <code>Type :: Type</code>
формы полиморфизма	высшие ранги и импредикативность (first-class), higher-kinded, GADT, продвижение данных, полиморфные кайнды
реализация	мономорфизация, стирание, гибрид, witness table — у каждого своя цена; словари вернутся в главе 3
представления	boxed/unboxed, lifted/unlifted, <code>TYPE r</code> ; связыватели не бывают полиморфны по представлению
• Домашка fr2-02: вектор с длиной в типе и типизированный <code>Expr</code>; формулы на уровне типов, <code>hzip</code>, <code>Tagged</code>, числа и пара Чёрча; тип <code>error</code> • На следующем занятии: ad-hoc полиморфизм — классы типов как словари, резолв инстансов, семейства типов — глава 3	

Материалы

Факультативно: эмуляция типовых приложений через Proxy

```
data Proxy a = Proxy                                -- фантомный параметр: тип без значения

id :: Proxy a -> a -> a

ghci> :t id (Proxy :: Proxy Int)
id (Proxy :: Proxy Int) :: Int -> Int

id (Proxy :: Proxy a) x = (x :: a)                  -- ScopedTypeVariables: сигнатура в паттерне
```

```
id :: proxy a -> a -> a                                -- прокси-тип оставлен полиморфным
id (_ :: proxy a) x = (x :: a)
```

Что почитать

- Конспект курса, глава 2 «Параметрический полиморфизм»
- Benjamin Pierce, «Types and Programming Languages», главы 23 (System F) и 29 (кайнды) — must-read по первой секции
- Brent Yorgey et al., [Giving Haskell a Promotion](#) (2012) — откуда взялись DataKinds
- Richard Eisenberg, Simon Peyton Jones, [Levity Polymorphism](#) (2017) — вся третья секция в одной статье
- Alejandro Serrano et al., [A Quick Look at Impredicativity](#) (2020) — как GHC 9.2 выводит импредикативные типы
- John Launchbury, Simon Peyton Jones, [Lazy Functional State Threads](#) (1994) — ST-trick
- [Implementing Swift Generics](#) (LLVM Developers' Meeting, 2017) — witness tables из первых рук
- Олег Киселёв, [Boehm–Berarducci encoding](#) — типы высшего ранга и кодирование данных функциями